Facultad de Astronomía, Matemática, Física y Computación

# LENGUAJES FORMALES Y COMPUTABILLIDAD

Pedro Villar

# 1 Guia 1: Notación y Conceptos Básicos

**Ejercicio 1:** Entender en forma precisa que conjunto se esta denotando en cada uno de los siguientes casos

(a) $\{x \in \mathbf{N} : x = 1 \text{ o } x \geq 5\}$
(b) $\{x : x \in \mathbf{R} \text{ y } x^2 \geq 100\}$
(c) $\{x : x = 100\}$
(d) $\{x^2 + 1 : x \in \omega\}$
(e) $\{x + y + z : x, y, z \in \{1, 2\}\}$

**Punto (a)** $\{x \in \mathbb{N} : x = 1$ **o** $x \geq 5\}$ El conjunto de todos los números naturales $x$ tal que $x$ es igual a $1$ o $x$ es mayor o igual a $5$. Por lo tanto está formado por el $1$ y todos los números a partir del $5$:

$$\{1, 5, 6, 7, 8, 9, ...\}$$

**Punto (b)** $\{x : x \in \mathbb{R}$ **y** $x^2 \geq 100\}$ El conjunto de todos los números reales $x$ tal que $x^2$ es mayor o igual a $100$. Esto es, el conjunto de todos los números reales $x$ tal que $x$ es menor o igual a $-10$ o $x$ es mayor o igual a $10$. Por lo tanto está formado por todos los números menores o iguales a $-10$ y todos los números mayores o iguales a $10$:

$$\{x \in \mathbb{R} : x \leq -10 \text{ o } x \geq 10\}$$

**Punto (c)** $\{x : x = 100\}$ El conjunto de todos los elementos $x$ tal que $x$ es igual a $100$. Por lo tanto, el conjunto formado por el número $100$:

$$\{100\}$$

**Punto (d)** $\{x^2 + 1 : x \in \omega\}$ El conjunto de todos los elementos de la forma $x^2 + 1$ tal que $x$ es un número natural o el cero. Por lo tanto, el conjunto formado por los números de la forma $n^2 + 1$ donde $n$ es un número natural o el cero:

$$\{1, 2, 5, 10, 17, ...\}$$

**Punto (e)** $\{x + y + z : x, y, z \in \{1, 2\}\}$ El conjunto de todos los elementos de la forma $x + y + z$ tal que $x$, $y$ y $z$ son números iguales a $1$ o $2$. Por lo tanto, el conjunto formado por los números de la forma $a + b + c$ donde $a$, $b$ y $c$ son números iguales a $1$ o $2$:

$$\{3, 4, 5, 6\}$$

**Ejercicio 2:** V o F o I, justifique.

(a) $\{x.y : x, y \in \omega\} = \omega$
(b) $|\{x.y : x, y \in \omega \text{ y } 1 \leq x, y \leq 5\}| = 25$
(c) Dados $A, B \subseteq \omega$, se tiene que $\{a \in A \text{ y } b \in B : a + b = 1000\} \subseteq A \times B$
(d) $\{a \in \mathbf{N}, a \geq 3\} \subseteq \omega$
(e) $\{x + 1 : x \in \{1, 2, 3\}\} = \{1, 2, 3, 4\}$

**Punto (a) - Impreciso** El lado izquierdo de la igualdad usa una notación con una alteración que la vuelve confusa. En la notación de conjuntos por comprensión, la parte a la izquierda de los dos puntos (:) debe indicar claramente qué tipo de objeto se está coleccionando. Al escribir $x, y$ en lugar de un par ordenado $(x, y)$ o una variable individual, se genera ambigüedad.

**Punto (b) - Falso** A nivel sintáctico, está escrito correctamente. Representa la cantidad de elementos del conjunto formado por los productos $x.y$ donde $x$ e $y$ son números naturales entre $1$ y $5$. Pero si calculamos a mano todas las multiplicaciones posibles, la cardinalidad de dicho conjunto es $14$.

**Punto (c) - Impreciso** La notación del conjunto de la izquierda tiene un error sintáctico confuso. Escribir condiciones separadas por la letra $y$ a la izquierda de los dos puntos hace que no quede claro qué tipo de objeto matemático se está definiendo. (*coincide con ejemplo 6 del archivo Bases de Tómbola.pdf*)

**Punto (d) - Impreciso** Tiene un error sintáctico, está utilizando una coma en lugar de dos puntos o una barra para separar los elementos de la condicion de pertenencia. Convierte la expresión en ambigua.

**Punto (e) - Falso** Si tomamos $x \in \{1, 2, 3\}$ y estamos evaluando una sola expresión $x + 1$, entonces el conjunto formado por los elementos de la forma $x + 1$ donde $x$ es un número igual a $1$, $2$ o $3$ es $\{2, 3, 4\}$, no $\{1, 2, 3, 4\}$.

**Lema 1.** *Sean $f$ y $g$ funciones. Entonces $f = g$ sii $D_f = D_g$ y para cada $x \in D_f$ se tiene que $f(x) = g(x)$*

.

**Ejercicio 3:** Pruebe el lema anterior (en el apunte esta probado)

Hay que probar que $f = g \Leftrightarrow Dom(f) = Dom(g)$.

$\Rightarrow$ Si $f = g$, quiere decir que todo elemento de $f$ es igual a un elemento de $g$ y viceversa por *definición de igualdad de conjuntos*.

$$f = \{(x, f(x)) : x \in D_f\}$$
$$g = \{(x, g(x)) : x \in D_g\}$$

- El conjunto de las primeras coordenadas deben ser identicas por igualdad de conjuntos.
- Además para cualquier $x \in D_f$, existe un único $y$ tal que $(x, y) \in f$, como $f = g$, $(x, y) \in g$ y por lo tanto $f(x) = y = g(x)$. Entonces $f$ y $g$ tienen el mismo dominio, es decir $D_f = D_g$.

$\Leftarrow$ Supongamos que $D_f = D_g$ y $\forall x \in D_f, f(x) = g(x)$. Tenemos que probar doble inclusión ya que buscamos llegar a que $f = g$ y ambos son conjuntos.

$f \subseteq g$ sea $(a, b) \in f$, entonces $a$ está en el dominio de $f$ y ademas $f(a) = b$, como $f(x) = g(x)$ para todo $x$ en el dominio de $f$, entonces $g(a) = b$ y por lo tanto $(a, b) \in g$, sin perdida de generalidad entonces, todo elemento de $f$ es un elemento de $g$ y por lo tanto $f \subseteq g$.

$g \subseteq f$ sea $(a, b) \in g$, entonces $a$ está en el dominio de $g$ y ademas $g(a) = b$, como $f(x) = g(x)$ para todo $x$ en el dominio de $g$, entonces $f(a) = b$ y por lo tanto $(a, b) \in f$, sin perdida de generalidad entonces, todo elemento de $g$ es un elemento de $f$ y por lo tanto $g \subseteq f$.

**Ejercicio 4:** V o F o I, justifique.

(a) Si

$$\begin{array}{rcl} f : \mathbf{N} & \to & \omega \\ x & \to & x^3 \end{array} \qquad \begin{array}{rcl} g : \mathbf{N} & \to & \mathbf{R} \\ x & \to & x^3 \end{array}$$

entonces $f = g$

(b) Si

$$\begin{array}{rcl} f : \mathbf{N} & \to & \omega \\ x & \to & x^3 \end{array} \qquad \begin{array}{rcl} g : \mathbf{N} & \to & \omega \\ x & \to & x^4/x \end{array}$$

entonces $f = g$

(c) Si $f$ es una funcion y $z \in D_f$, entonces $Ti(z) =$ CONJUNTO

(d) $\text{Dom}((1, 2)) = \{1\}$

(e) $\text{Dom}(\{(1, 2)\}) + 1 = 2$

(f) Si $f$ es una funcion, entonces $D_f = \{a : (a, b) \in f\}$

(g) Si $f : A \to B$, entonces $D_f \subseteq A$

(h) Si $f : A \to B$, entonces $I_f = B$

(i) Si $f$ es una función y $g \subseteq f$, entonces $g$ es una función

**Punto (a) - Verdadero** El lema del criterio de igualdad de funciones demostrado en el ejercicio anterior nos dice que dos funciones $f$ y $g$ son iguales si y solo si sus dominios coninciden y para cada elemento $x$ del dominio, $f(x) = g(x)$. En este caso el dominio de ambas funciones es $\mathbb{N}$ y, al evaluarlas, ambas devuelven $x^3$. La convencion notacional $f : A \to B$ nos dice que $B$ es simplemente un *conjunto de llegada*, cualquiera que contiene a la imagen, y este no determina la identidad de la función. Dado que las funciones como conjuintos de pares ordenados son iguales, entonces $f = g$.

**Punto (b) - Verdadero** Ambas tienen el mismo dominio $\mathbb{N}$, como eso denota los naturales a partir del $1$, $\omega$ denota los naturales mas el $0$, es decir, el valor $x$ nunca va a ser $0$ y por lo tanto la división en la regla de $g$ siempre estará definida y devuelve $x^3$. Entonces ambas funciones tienen el mismo dominio y devuelven lo mismo para cada valor de $x$, por lo tanto $f = g$.

**Punto (c) - Falso** Los elementos del dominio pueden ser de cualquier tipo definido dentro de la matemática orientada a objetos.

**Punto (d) - Impreciso** La operación de dominio está definida para aplicarse sobre funciones, es decir, sobre conjuntos de pares ordenados que cumplen la propiedad de función. El objeto $(1, 2)$ es una $2$-upla. Es una imprecisión decir que el dominio de $(1, 2)$ es $\{1\}$, ya que el dominio no está definido para objetos que no son funciones.

**Punto (e) - Impreciso** La operación de dominio está bien definida, pero esta da como resultado un conjunto de elementos, y al reemplazar en la expresión, nos queda la suma de un conjunto y un número, lo cual es una imprecisión.

**Punto (f) - Impreciso** Es impreciso puesto que hay un error de sintaxis en la definición por comprensión del conjunto. La variable $b$ es libre, es decir, no fue cuantificada.

**Punto (g) - Verdadero** Si $f : A \to B$, entonces $D_f = A$ y se cumple entonces la doble inclusión y por lo tanto $D_f \subseteq A$ y $A \subseteq D_f$.

**Punto (h) - Falso** Por Convención Notacional, $B$ es simplemente un conjunto de llegada, no es necesariamente $I_f$.

**Punto (i) - Verdadero** Por definición, una función $f$ es un conjunto de pares ordenados que cumplen cierta propiedad. Si $g \subseteq f$, $g$ es simplemente un subconjunto de esos pares ordenados. Si $f$ era función, es imposible que tomando un subconjunto de sus pares ordenados se pierda la propiedad de función, por lo que $g$ también es una función.

**Ejercicio 5:** V o F o I, justifique.

(a) La funcion $x + 1$ es $\emptyset$-mixta

(b) La función

$$\begin{array}{rcl} \left\{(x, \alpha) \in \omega \times \{\#, \&, @\}^* : |\alpha|_{\#} = 0\right\} & \to & \omega \\ (x, \alpha) & \to & |\alpha| . x \end{array}$$

es $\{\&, @\}$-mixta

(c) $f$ es $\Sigma$-mixta si existen $n, m \geq 0$, tales que $D_f \subseteq \omega^n \times \Sigma^{*m}$ y $I_f \subseteq \omega \cup \Sigma^*$

(d) Sea $\begin{array}{rcl} f : \omega & \to & \omega \\ x & \to & C_2^{1,0} \end{array}$. Entonces $f(5) = 2$

**Punto (a) - Impreciso** La expresión $x + 1$ por sí sola es ambigua porque no queda claro cuál es el dominio de dicha función.

**Punto (b) - Verdadero** Verdadero, cumple con las dos condiciones de que sea $\Sigma$-mixta.

**Punto (c) - Falso** La función mixta debe devolver exclusivamente símbolos o palabras. De la forma en la que está escrita, la función podría devolver valores mezclados.

**Punto (d) - Falso** El retorno de dicha función, no es un número sino otro tipo de función. Por lo tanto la función devuelve conjuntos, y decir que $f(5) = 2$ es falso, ya que $f(5)$ no es un número sino un conjunto de pares ordenados.

**Ejercicio 6:** Hacer

(a) De que tipo es cada una de las siguientes funciones

(i) $C_{\varepsilon}^{1,2}$

(ii) $\left\{(x,\alpha) \in \omega \times \{\#, \&, @\}^* : |\alpha|_{\#} = 0\right\} \rightarrow \omega$

$(x,\alpha) \rightarrow |\alpha| .x$

(iii) $Id_{\omega}$

(iv) $Id_{\Sigma^*}$

(v) $\Sigma^* \rightarrow \omega$

$\alpha \rightarrow |\alpha|$

(vi) $\{(\varepsilon,\varepsilon)\} \rightarrow \{\varepsilon\}$

$(\varepsilon,\varepsilon) \rightarrow \varepsilon$

(vii) $\{\Diamond\} \rightarrow \omega$

$\Diamond \rightarrow 0$

(b) Que significa la frase

- la relacion "$f$ es una funcion de tipo $(n, m, s)$" no depende del alfabeto $\Sigma$

Intente expresar esto en forma matematica

**Punto a)** El tipo de una función $\Sigma$-mixta se denota como $(n, m, s)$, donde $n$ es la cantidad de variables numéricas que toma, $m$ es la cantidad de variables alfabéticas (palabras) que toma, y $s$ es $\#$ si la imagen devuelve un número ($\omega$) o $*$ si devuelve una palabra ($\Sigma^*$).

(i) **Tipo:** $(1, 2, *)$.

(ii) **Tipo:** $(1, 1, \#)$.

(iii) **Tipo:** $(1, 0, \#)$.

(iv) **Tipo:** $(0, 1, *)$.

(v) **Tipo:** $(0, 1, \#)$.

(vi) **Tipo:** $(0, 2, *)$.

(vii) **Tipo:** $(0, 0, \#)$.

**Punto b)** Significa que la clasificación del tipo de una función depende exclusivamente de su estructura de conjunto, es decir, la cantidad de números y palabras que recibe como entrada, y si devuelve un número o una palabra como salida. Es decir, una cadena de caracteres es vista como el objeto matemático *palabra*, mas allá de qué alfabeto vengan esas palabras. Entonces para decir que algo no depende de otra cosa, podemos expresar como que dos cosas que miran de forma distinta al mismo resultado, llegan al mismo resultado sin importar qué herramientas usen.

Para toda función $f \neq \emptyset$, si existen alfabetos $\Sigma$ y $\Gamma$ tales que $f$ es de tipo $(n, m, s)$ respecto al alfabeto $\Sigma$, y $f$ es de tipo $(n', m', s')$ respecto al alfabeto $\Gamma$, entonces obligatoriamente $n = n'$, $m = m'$ y $s = s'$.

**Ejercicio 7:** V o F o I, justifique

(a) Si $D_f = \omega$, entonces $f \circ \{(x,x) : x \in \omega\} = f$
(b) Si $D_f = \omega$, entonces $f \circ x = f$
(c) $Pred = Pred \circ (Pred \circ Suc)$
(d) $Pred \circ (Suc \circ Pred) = Pred$
(e) $Pred \circ (Suc \circ \{(x,x) : x \in \mathbf{N}\}) = Pred \circ Suc$
(f) $\emptyset \circ f = f \circ \emptyset = \emptyset$ cualquiera sea la funcion $f$
(g) Sea $\Sigma$ un alfabeto finito. Si $x_1, x_2, x_3, x_4, x_5 \in \omega$ se tiene que $(Suc \circ p_2^{5,0})(x_1, x_2, x_3, x_4, x_5) = x_3$
(h) Sea $\Sigma$ un alfabeto finito. Entonces $Suc \circ Pred = p_1^{1,0}$
(i) $Suc \circ x = Suc$
(j) $Suc \circ 4 = 5$
(k) Sea $\Sigma$ un alfabeto finito. Entonces $\emptyset = Pred \circ C_0^{0,0}$

**Punto (a) - Verdadero** El conjunto $g = (x,x) : x \in \omega$ es la función identidad sobre $\omega$ (es decir, $Id_\omega$). Y esto era de la siguiente manera: su dominio es $\omega$ y para cada $x \in \omega$, devuelve $x$.

- $D_{f \circ g} = \{e \in D_g : g(e) \in D_f\} = \{e \in \omega : e \in \omega\} = \omega$. Y como el enunciado nos dice que $D_f = \omega$, los dominios coinciden.
- Al evaluar: $(f \circ g)(x) = f(g(x)) = f(x)$. Como tienen el mismo dominio y devuelven los mismos valores, por el Lema de Igualdad de Funciones, $f \circ g = f$.

**Punto (b) - Impreciso** La operación de composición ($\circ$) está definida exclusivamente entre dos funciones.

**Punto (c) - Verdadero**

- $D_{Suc} = \omega$ y $D_{Pred} = \mathbb{N}$, entonces $D_{Pred \circ Suc} = \{e \in D_{Suc} : Suc(e) \in D_{Pred}\} = \{e \in \omega : e + 1 \in \mathbb{N}\}$.
- $(Pred \circ Suc)(x) = Pred(x+1) = x$. Por lo tanto, $Pred \circ Suc = Id_\omega$.
- $D_{Pred \circ Id_\omega} = \{x \in D_{Id_\omega} : Id_\omega(x) \in D_{Pred}\} = \{x \in \omega : x \in \mathbb{N}\} = \mathbb{N}$.
- $(Pred \circ Id_\omega)(x) = Pred(x)$. Como el dominio resultante es $\mathbb{N}$ (que es exactamente $D_{Pred}$) y devuelve los mismos valores, la igualdad se cumple.

**Punto (d) - Verdadero** De forma análoga al punto anterior:

- $D_{Suc \circ Pred} = \{x \in D_{Pred} : Pred(x) \in D_{Suc}\} = \{x \in \mathbb{N} : x - 1 \in \omega\} = \mathbb{N}$.
- $(Suc \circ Pred)(x) = Suc(x-1) = x$. Esto significa que $Suc \circ Pred = Id_\mathbb{N}$.
- $D_{Pred \circ Id_\mathbb{N}} = \{x \in D_{Id_\mathbb{N}} : Id_\mathbb{N}(x) \in D_{Pred}\} = \{x \in \mathbb{N} : x \in \mathbb{N}\} = \mathbb{N}$. Como el dominio coincide con el de $Pred$ y el resultado de evaluarla da $Pred(x)$, la igualdad es verdadera.

**Punto (e) - Falso** Se puede ver que los dominios de ambos lados de la igualdad son distintos, el lado izquierdo no incluye al $0$ mientras que el derecho sí.

**Punto (f) - Verdadero** Usando el lema: $f \circ g \neq \emptyset \iff I_g \cap D_f \neq \emptyset$.

- Para $\emptyset \circ f$: La función izquierda es $\emptyset$, por lo que su dominio es $\emptyset$. La intersección $I_f \cap D_\emptyset$ será $\emptyset$.
- Para $f \circ \emptyset$: La función derecha es $\emptyset$, cuya imagen es $\emptyset$. La intersección $I_\emptyset \cap D_f$ será $\emptyset$. En ambos casos la intersección es vacía, por lo que la composición resultante es necesariamente la función vacía $\emptyset$.

**Punto (g) - Falso**

**Punto (h) - Falso** A simple vista se ve que el dominio no coincide y por lo tanto las funciones no pueden ser iguales:

- El dominio de la proyección $p^1_{1,0}$ es $\omega^1 \times \Sigma^{*0}$, es decir, $\omega$.
- $Suc \circ Pred$: por definición es $\{x \in D_{Pred} : Pred(x) \in D_{Suc}\}$. Sabemos que $D_{Pred} = \mathbb{N}$ y $D_{Suc} = \omega$. Nos queda $x \in \mathbb{N} : x - 1 \in \omega$. Dado que para cualquier $x \in \mathbb{N}$ su predecesor es al menos 0 (que pertenece a $\omega$), el dominio resultante es $\mathbb{N}$.

**Punto (i) - Impreciso** Se está haciendo composición de una función y un objeto matemático que no es un conjunto, por lo que es ambiguo.

**Punto (j) - Impreciso** Igual que el punto anterior.

**Punto (k) - Verdadero** Usando el lema devuelta $f \circ g = \emptyset \iff I_g \cap D_f = \emptyset$.

- La función de la derecha ($g$) es la constante $C^0_{0,0}$. Por definición, su imagen es siempre el número de arriba, es decir, $I_C = 0$.
- La función de la izquierda ($f$) es $Pred$. Su dominio son los naturales a partir del 1, es decir, $D_{Pred} = \mathbb{N}$. Hacemos la intersección: $0 \cap \mathbb{N}$. Como el 0 no pertenece a $\mathbb{N}$, la intersección da como resultado el conjunto vacío ($\emptyset$). Al cumplirse la doble implicación del lema, la composición entera resulta en la función vacía $\emptyset$.

**Ejercicio 8:** Pruebe que $f \circ g \neq \emptyset$ si y solo si $I_g \cap D_f \neq \emptyset$ (esta probado en el apunte). Notese que este resultado nos dice que que muchas veces sucedera que $f \circ g = \emptyset$.

FUNCIONES DE LA FORMA $[f_1, ..., f_n]$

Dadas funciones $f_1, ..., f_n$, con $n \geq 2$, definamos la funcion $[f_1, ..., f_n]$ de la siguiente manera:

$$D_{[f_1,...,f_n]} = D_{f_1} \cap ... \cap D_{f_n}$$
$$[f_1, ..., f_n](e) = (f_1(e), ..., f_n(e))$$

Notese que $I_{[f_1,...,f_n]} \subseteq I_{f_1} \times \cdots \times I_{f_n}$. Por conveniencia notacional (que el lector entendera mas adelante) definiremos $[f_1] = f_1$. Es decir que hemos definido para cada succecion de funciones $f_1, ..., f_n$, con $n \geq 1$, una nueva funcion la cual denotamos con $[f_1, ..., f_n]$.

Al ser una doble implicación, tenemos que probar ambas direcciones.

**[⇒]** Asumimos como hipótesis que nuestra función compuesta no es el conjunto vacío ($f \circ g \neq \emptyset$). Entonces si una función no es el conjunto vacío, puedo decir que su dominio tampoco lo es, $D_{f \circ g} \neq \emptyset$. Por la definición del dominio de la composición, **existe al menos un elemento** $e \in D_g$ **tal que** $g(e) \in D_f$. Ahora bien, como $e$ es un elemento del dominio de $g$, el resultado de evaluarlo, es decir $g(e)$, pertenece por definición a la imagen de $g$. Con esto, encontramos un elemento que sabemos que está en $I_g$ y que también está en $D_f$, por lo tanto la intersección no es vacía.

**[⇐]** Asumimos ahora que la intersección no es vacía, entonces podemos tomar algún elemento fijo que pertenezca tanto a $I_g$ y a $D_f$, le pongo $x$. Como $x \in I_g$, entonces hay un elemento $e \in D_g$ tal que $g(e) = x$, y dijimos que $x \in D_f$ entonces $g(e) \in D_f$ entonces encontramos un elemento que cumple la condición de pertenencia al dominio de la composición, por lo tanto $f \circ g \neq \emptyset$.

**Ejercicio 9:** V o F o I, justifique

(a) Sea $\Sigma$ un alfabeto y supongamos $\# \in \Sigma$. Entonces

$$p_4^{2,3} \circ [p_1^{1,1}, p_1^{1,1}, p_2^{1,1}, C_{\#\#}^{1,1}, p_2^{1,1}] = C_{\#\#}^{1,1}$$

(b) Si $f : \omega^2 \to \omega$, entonces $f = f \circ [x, y]$

(c) $[p_2^{2,3}, Suc] = \emptyset$

(d) Supongamos $f_i : \omega \to \omega$, para $i \in \{1, ..., n\}$, con $n \geq 2$. Entonces $I_{[f_1,...,f_n]} = I_{f_1} \times \cdots \times I_{f_n}$

**Punto (a) - Verdadero** Primero voy a ver lo que está a la derecha: $g = [p_{1,1}^1, p_{1,1}^1, p_{1,1}^2, C_{1,1}^{\#\#}, p_{1,1}^2]$ por definición sé que el dominio de esto , es la intersección de los dominios de todas las funciones que la componen. Como todas son funciones del tipo $(1, 1, s)$, el dominio de $g$ es exactamente $\omega \times \Sigma^*$. Al aplicar la función $g$ en un par $(x, \alpha)$, obtenemos una $5$-upla aplicando cada función a la entrada, y esta salida es una tupla de $2$ números y $3$ palabras por las funciones que tenemos, $g(x, \alpha) = (x, x, \alpha, \#\#, \alpha)$.

1. **Primera función ($p_{1,1}^1$):** Es una proyección. Toma el par $(x, \alpha)$ y devuelve la coordenada 1. Por lo tanto, devuelve $x$.
2. **Segunda función ($p_{1,1}^1$):** Es exactamente la misma proyección. Por lo tanto, también devuelve $x$.
3. **Tercera función ($p_{1,1}^2$):** Es la proyección de la coordenada 2. Al darle el par $(x, \alpha)$, devuelve la palabra. Por lo tanto, devuelve $\alpha$.
4. **Cuarta función ($C_{1,1}^{\#\#}$):** Es una función constante. Las funciones constantes ignoran la entrada que les des y devuelven siempre lo que tienen en su superíndice. Al darle $(x, \alpha)$, ignora todo y devuelve $\#\#$.
5. **Quinta función ($p_{1,1}^2$):** Vuelve a ser la proyección de la coordenada 2. Devuelve $\alpha$.

Esto coincide con la función que está a la izquierda, ya que la función $p_{2,3}^4$ toma $2$ números y $3$ palabras, y devuelve la coordenada $4$, que es exactamente $\#\#$. Entonces, para cualquier entrada $(x, \alpha)$, la composición completa devuelve la palabra $\#\#$, que es exactamente lo mismo que hace la función constante $C_{1,1}^{\#\#}$.

**Punto (b) - Impreciso** $[x, y]$ es un objeto raro, ya que el constructor de funciones está definido para funciones y hay números.

**Punto (c) - Verdadero** Por definición, el dominio de la función resultante va a ser la intersección de dominios de las componentes.

- El dominio de la proyección $p_{2,3}^2$ es $\omega^2 \times \Sigma^{*3}$, es decir, el conjunto de todas las 5-uplas formadas por 2 números y 3 palabras.
- El dominio de la función $Suc$ es $\omega$.

Acá las tuplas y los números son objetos matemáticos de categorías disjuntas, la intersección es vacía ($D = \emptyset$). Si una función tiene dominio vacío, entonces es vacía $\emptyset$.

**Punto (d) - Falso** Tomo como contraejemplo $n = 2$ y dos funciones identidad: $f_1 = Id_\omega$ y $f_2 = Id_\omega$. El producto cartesiano de sus imágenes es $I_{f_1} \times I_{f_2} = \omega \times \omega$, o sea, todos los pares posibles de números. De igual manera, si evaluo la tupla: $Id_\omega, Id_\omega = (Id_\omega(x), Id_\omega(x)) = (x, x)$. Esto me dice que la imagen real de la función tupla es $I_{[f_1,f_2]} = (x, x) : x \in \omega$, es decir, solo los pares donde ambos números son iguales. Como $(x, x) : x \in \omega \neq \omega \times \omega$.

**Ejercicio 10:** V o F o I, justifique.

(a) Una función $f$ es inyectiva si $f(x) = f(y)$ cada vez que $x = y$

(b) $F : A \to B$ es suryectiva sii para cada $a \in A$ existe un $b \in B$ tal que $b = F(a)$

**Punto (a) - Falso** Esto es lo mismo que decir que todas las funciones son inyectivas.

**Punto (b) - Falso** Lo que dice el enunciado luego del si y solo si, es simplemente lo que significa $A \to B$, no hace que sea suryectiva.

**Ejercicio 11:** Hacer:
(a) Dar una biyeccion entre $\mathbf{N}$ y $\omega$. Idem entre $\omega$ y $\{x \in \omega : x \text{ es par}\}$
(b) Dar una funcion inyectiva de $\omega^2$ en $\omega$
(c) Dar una funcion sobreyectiva de $\omega$ en $\omega^5$

**Punto (a)** Busco una función inyectiva y sobreyectiva que asigne a cada número natural un número en $\omega$ .

$$f : \mathbb{N} \to \omega$$

$$x \mapsto x - 1$$

- **Es inyectiva:** Si $x - 1 = y - 1$, sumando 1 a ambos lados, $x = y$,.
- **Es sobreyectiva:** Para cualquier $y \in \omega$, existe un $x \in \mathbb{N}$ ($x = y + 1$) tal que $f(x) = y$.

Para el segundo caso

$$g : \omega \to x \in \omega : x \text{ es par}$$

$$x \mapsto 2 \cdot x$$

- **Es inyectiva:** Si $2x = 2y$, dividiendo por 2, $x = y$,.
- **Es sobreyectiva:** Por definición de número par, para cualquier $y$ par existe un natural (o cero) $k \in \omega$ tal que $y = 2k$,.

**Punto (b)** Aquí necesitamos codificar un par de números $(x, y)$ en un único número de forma tal que no de problemas.

$$h : \omega^2 \to \omega$$

$$(x, y) \mapsto 2^x \cdot 3^y$$

Se tiene que $h(x, y) = h(a, b)$. Esto significa que $2^x \cdot 3^y = 2^a \cdot 3^b$. Por el teorema de factorización única de primos, la única forma de que estos números sean iguales es que los exponentes de las respectivas bases primas sean exactamente los mismos.

**Punto (c)** Ahora debe tomar un único número de $\omega$ y escupir una 5-upla en $\omega^5$. Para que sea sobreyectiva, **absolutamente todas las 5-uplas posibles** deben poder ser generadas por la función al menos una vez.

$$F : \omega \to \omega^5$$

$$x \mapsto ((x)_1, (x)_2, (x)_3, (x)_4, (x)_5)$$

Tomo un elemento cualquiera del conjunto de llegada, es decir, una 5-upla arbitraria $(a, b, c, d, e) \in \omega^5$. Para ver si existe algún número $x \in \omega$ que, al dárselo a la función, nos devuelva esa tupla exacta, basta con construir el número mediante la multiplicación de los 5 primeros primos elevados a esos elementos:

$$x = 2^a \cdot 3^b \cdot 5^c \cdot 7^d \cdot 11^e$$

**Lema 3.** *Supongamos $f : A \to B$ y $g : B \to A$ son tales que $f \circ g = Id_B$ y $g \circ f = Id_A$. Entonces $f$ y $g$ son biyectivas, $f^{-1} = g$ y $g^{-1} = f$.*

**Ejercicio 12:** Haga una prueba del lema anterior (esta probado en el apunte)

La táctica sería probar que $f$ es biyectiva y que su inversa es $g$. Para luego decir que por simetría, aplica para $g$ también.

**$f$ es inyectiva** Para que $f$ sea inyectiva, tenemos que demostrar que si $f(a) = f(b)$, entonces obligatoriamente $a = b$.

Supongamos que tomamos dos elementos $a, b \in A$ y vemos que $f(a) = f(b)$. Si dos objetos son el mismo, aplicarles la máquina $g$ nos dará el mismo resultado. Entonces, apliquemos $g$ a ambos lados: $g(f(a)) = g(f(b))$ que es lo mismo que $(g \circ f)(a) = (g \circ f)(b)$. Pero el enunciado nos dice que $g \circ f = Id_A$, entonces reemplazando nos queda $Id_A(a) = Id_A(b)$, lo cual significa que $a = b$.

**$f$ es suryectiva** Para que $f$ sea suryectiva, su imagen tiene que ser todo el conjunto de llegada $B$ ($I_f = B$).

Sea $b \in B$ un elemento cualquiera. Como $f \circ g = Id_B$. Esto significa que para nuestro elemento $b$, se cumple que $(f \circ g)(b) = Id_B(b)$, que es lo mismo que decir $f(g(b)) = b$, entonces $b$ puede expresarse como $f$ de algo, por lo tanto $b \in Im_f$, y por lo tanto $f$ alcanza a todos los elementos de $B$.

Como $f$ es inyectiva y sobreyectiva, entonces es biyectiva y sabemos por definición que existe su función inversa $f^{-1} : B \to A$. Su dominio es exactamente $B$.

**$f^{-1} = g$** Tenemos que buscar que tengan el mismo dominio y devuelvan los mismos valores.

Primero que nada el dominio de $f^{-1}$ es $B$ y el dominio de $g$ también es $B$. Ahora los valores, sea $b \in B$. Por la definición de función inversa, $f^{-1}(b)$ es el único elemento $a \in A$ que cumple que $f(a) = b$. Y un poco antes ya dijimos que $f(g(b)) = b$, por lo tanto es elemento único tiene que ser obligatoriamente $g(b)$. En consecuencia $f^{-1}(b) = g(b)$.

(a) Un conjunto $S$ es $\Sigma$-mixto sii $S = D_f$ para alguna función $\Sigma$-mixta $f$
(b) $\{(1, 2, \varepsilon), (1, 2)\}$ es un conjunto $\Sigma$-mixto, cualesquiera sea el alfabeto finito $\Sigma$

**Punto (a) - Verdadero**

**Punto (b) - Falso** Para que un conjunto sea $\Sigma$-mixto, su cantidad de números y palabras debe ser fija obligatoriamente. En este caso se chocan las cantidades.

**Ejercicio 15:** V o F o I, justifique

(a) La expresion
$$|\alpha\#@@| + x$$
no es lambdificable con respecto a $\{\#, \%\}$

(b) La expresion
$$x + 1 = 1/3$$
es lambdificable con respecto a $\Sigma$ cualesquiera sea $\Sigma$

(c) La expresion
$$\lambda x[x^2] + (|\beta| + 1)$$
es lambdificable con respecto a $\Sigma$ cualesquiera sea $\Sigma$

**Punto (a) - Falso** Sí es lambdificable para cualquier alfabeto, siempre devuelve un número.

**Punto (b) - Verdadero** Nunca generaría nada, entonces si es lambdificable para cualquier alfabeto.

**Punto (c) - Verdadero** Si busco lambdificar todo eso, quedaría algo de la forma FUNCIÓN + NÚMERO, lo cual nunca genera nada, nunca da resultados que violen la regla de lambda, por lo tanto, es lambdificable con respecto a cualquier alfabeto.

# 2 Guía 2: Dos codificaciones importantes

**Ejercicio 7:** Sea $\Sigma = \{\%, !, @\}$ y sea $\leq$ el orden total sobre $\Sigma$ dado por $\% < ! < @$ (es decir que aquí $a_1 = \%$, $a_2 = !$ y $a_3 = @$). Escriba los primeros elementos de la lista y describa para este caso particular la función $s^{\leq}$, sin hablar de los $a_i$, i.e. solo haciendo referencia a los símbolos $\%, !, @$.

Para armar la lista de elementos, se hace por longitud de menor a mayor y luego por orden lexicográfico:

[ $\varepsilon$, %, !, @, % %, %!, %@, ! %, !!, !@ , @ %, @!, @@, ...]

- **Casi 1: la palabra está formada exclusivamente por el símbolo máximo**, es decir el caso donde se acabaron todas las combinaciones posibles para una longitud específica:

$$s_{\leq}(@^m) = \%^{m+1}$$

- **Caso 2: la palabra es interna, no está formada exclusivamente por el símbolo máximo**, acá hay que buscar el primer símbolo desde la derecha que no sea el máximo, lo subimos a su siguiente y lo rellenamos todo lo que está a su derecha con el primer símbolo.

$$s_{\leq}(\alpha\,\%@^m) = \alpha!\,\%^m$$

por ejemplo para buscar el sucesor de ! %@, en este caso $\alpha$ es !, el símbolo a subir es el $\%$, y hay un $@$ a la derecha. El $\%$ sube a ! y el $@$ se reemplaza por $\%$, entonces el resultado es !! $\%$.

$$s_{\leq}(\alpha!@^m) = \alpha@\,\%^m$$

**Ejercicio 8:** Pruebe para el caso general que
(a) $\varepsilon \neq s^{\leq}(\alpha)$, para cada $\alpha \in \Sigma^*$
(b) Si $\alpha \neq \varepsilon$, entonces $\alpha = s^{\leq}(\beta)$ para algún $\beta$

Para las pruebas, voy a usar que $s_{\leq}((a_n)^m) = (a_1)^{m+1}$ y $s_{\leq}(\alpha a_i(a_n)^m) = \alpha a_{i+1}(a_1)^m$, con $i < n$.

**(a)** Probar que $\varepsilon \neq s_{\leq}(\alpha)$ para toda $\alpha \in \Sigma^*$: es como probar que la palabra $\epsilon$ vacía nunca es sucesor de ninguna palabra. Para ello basta con mirar la longitud de las palabras que escupe la máquina $s_{\leq}$ en sus dos casos posibles:

- En el primer caso, la máquina devuelve $(a_1)^{m+1}$ para algún $m \geq 0$. La longitud de esta palabra es $m+1$. Como el valor mínimo de $m$ es 0, la longitud resultante será siempre $\geq 1$.
- En el segundo caso, la máquina devuelve $\alpha a_{i+1}(a_1)^m$ para algún $m \geq 0$ y alguna palabra $\alpha$. La longitud de esta nueva palabra es $|\alpha| + 1 + m$. Como tanto $|\alpha|$ como $m$ son mayores o iguales a cero, la longitud total también será obligatoriamente $\geq 1$.

Como en absolutamente todos los casos posibles la palabra que devuelve la función $s_{\leq}$ tiene una longitud mayor o igual a 1, **es imposible que devuelva la palabra vacía $\varepsilon$** (cuya longitud es 0).

**(b)** Probar que si $\alpha \neq \varepsilon$, entonces $\alpha = s_{\leq}(\beta)$ para algún $\beta$: es como decir que toda palabra de la lista, excepto la primera, tiene uyn predecesor inmediato.

Como por hipótesis $\alpha \neq \varepsilon$, sabemos que su longitud es al menos 1. Acá podemos dividir en los dos casos de siempre pero al revés de $s_{\leq}$, ya que lo que buscariamos es como ir marcha atrás buscando ese $\beta$:

- El primer caso es cuando $\alpha$ está formada únicamente por el primer símbolo del alfabeto $a_1$. Es decir $\alpha = (a_1)^k$ con $k \geq 1$. Podemos reescribir el $k$ como $m+1$ (donde $m = k-1 \geq 0$). Y si tenemos $\alpha = (a_1)^{m+1}$, entonces por la definición de $s_{\leq}$, que esa palabra vino de $\beta = (a_n)^m$. Entonces acá vale la afirmación.

- El segundo caso es cuando $\alpha$ no está formada puramente por el primer símbolo, esto significa que buscando desde la derecha, tiene que haber al menos un símbolo que no sea el primero. Sea $a_j$ el primer símbolo distinto de $a_1$ que encontramos leyendo de derecha a izquierda. Todo lo que esté a la derecha de $a_j$ serán puras letras $a_1$ (una cantidad $m \geq 0$), y a su izquierda habrá un tramo de palabra cualquiera $\gamma$. Es decir la palabra será de la forma $\alpha = \gamma a_j (a_i)^m$. Para encontrar de dónde vino, aplico la regla de $s_\leq$ pero al revés. Le bajo un nivel a la letra $a_j$, pasa a ser $a_{j-1}$ y transformamos todos los primeros símbolos $a_1$ de la derecha en símbolos máximos $a_n$. Por lo tanto, esta palabra vino de la palabra $\beta = \gamma a_{j-1}(a_n)^m$. Entonces acá también vale la afirmación.

  Si el ejemplo es distinto de ver, supongamos que el alfabeto es $\{\%, !, @\}$, el símbolo máximo $a_n$ es @ y el mínimo $a_1$ es %. Tomemos la palabra $\alpha = @!\,\%\,\%$. Ahora hacemos la inversa, busco desde la derecha el primer símbolo que no sea el mínimo, es decir el !, lo bajo al anterior, es decir al %, y los dos mínimos que tenía a la derecha se transforman en máximos @@. Entonces la palabra anterior a esta es $\beta = @\,\%@@$. Y al aplicarle la función $s_\leq$ a $\beta$, se cumple que $s_\leq(@\,\%@@) = @!\,\%\,\%$, que es exactamente $\alpha$.

**Ejercicio 9:** (a) Convensace de que valen las siguientes ecuaciones

$$s^{\leq}(\varepsilon) = a_1$$
$$s^{\leq}(\alpha a_i) = \alpha a_{i+1},\ i < n$$
$$s^{\leq}(\alpha a_n) = s^{\leq}(\alpha) a_1$$

(b) Note que sucesivas aplicaciones de las ecuaciones anteriores determinan por completo el valor de $s^{\leq}$ en una palabra $\alpha$ previamente fijada. Explique esto con palabras.

**Punto (a)**

1. La palabra vacía $\varepsilon$ no tiene símbolos, por lo que su longitud es 0. En la definición formal, esto encaja en el primer caso $s_\leq((a_n)^m)$ tomando $m = 0$ (es decir, cero cantidad de letras máximas).

$$s_\leq((a_n)^0) = (a_1)^{0+1} = a_1^1 = \mathbf{a_1}$$

2. Si aplicamos la fórmula general para $m = 0$: $s_\leq(\alpha a_i (a_n)^0) = \alpha a_{i+1}(a_1)^0$. Como $(a_n)^0$ y $(a_1)^0$ son la palabra vacía $\varepsilon$, la fórmula se simplifica a $\alpha a_{i+1}$.

3. Si la palabra termina en el símbolo máximo ($a_n$), la ecuación afirma que ese símbolo se convierte en el mínimo ($a_1$) y el cálculo del sucesor se *traslada* a la parte izquierda de la palabra ($\alpha$).

   - Si $\alpha$ son puros símbolos máximos ($a_n^m$), entonces toda la palabra es $a_n^{m+1}$. Su sucesor es $a_1^{m+2}$. Usando la ecuación recursiva tendríamos $s_\leq(a_n^m)a_1 = a_1^{m+1}a_1 = a_1^{m+2}$.
   - Si $\alpha$ tiene algún símbolo no máximo ($\gamma a_i a_n^m$), entonces toda la palabra es $\gamma a_i a_n^{m+1}$. Su sucesor es $\gamma a_{i+1} a_1^{m+1}$. Usando la ecuación recursiva tendríamos $s_\leq(\gamma a_i a_n^m)a_1 = (\gamma a_{i+1} a_1^m)a_1 = \gamma a_{i+1} a_1^{m+1}$.

**Punto (b)** Si queremos calcular el sucesor de una palabra cualquiera $\alpha$, seguimos este proceso de toma de decisiones:

1. Si **la palabra está vacía ($\varepsilon$)**, la primera regla nos dice que devolvamos inmediatamente el primer símbolo del alfabeto ($a_1$) y el cálculo termina.

2. **Si la palabra no está vacía, miramos su última letra.**

3. **Si esa última letra no es el símbolo máximo** ($a_i < a_n$), la segunda regla nos dice que simplemente cambiemos esa letra por la que le sigue en el alfabeto ($a_{i+1}$) y dejemos el resto de la palabra ($\alpha$) intacta.

4. **Si esa última letra sí es el símbolo máximo ($a_n$),** la tercera regla nos indica que cambiemos esa última letra por el símbolo mínimo ($a_1$) y que repitamos todo el proceso evaluando al resto de la palabra que quedó a la izquierda ($\alpha$).

**Ejercicio 10:** Si $\leq$ es el orden de $\{@, \&\}$ dado por $@ < \&$, entonces $\#^{\leq}(@\&@\&@) = 2^4 + 2^4 + 2^2 + 2^2 + 1$ y $\#^{\leq}(@\&@\&@) = 2^5 + 2^3 + 1$

Usando $\#_{\leq}(\alpha) = i_k n^k + \cdots + i_0 n^0$, voy a buscar los índices, como la palabra tiene longitud $5$, los exponentes van a ir del $4$ al $0$.

- Posición 4: @, es decir $i_4 = 1$
- Posición 3: &, es decir $i_3 = 2$
- Posición 2: @, es decir $i_2 = 1$
- Posición 1: &, es decir $i_1 = 2$
- Posición 0: @, es decir $i_0 = 1$

La suma sería $1 \cdot 2^4 + 2 \cdot 2^3 + 1 \cdot 2^2 + 2 \cdot 2^1 + 1 \cdot 2^0 = 2^5 + 2^3 + 1$.

**Ejercicio 11:** Si $\leq$ es el orden de $\{@, \&\}$ dado por $@ < \&$, entonces $\#^{\leq}(\alpha@) = \#^{\leq}(\alpha).2 + 1$, para todo $\alpha \in \{@, \&\}^*$

Supongamos que $\alpha = a_{i_k} \ldots a_{i_0}$. Su código base es $\#_{\leq}(\alpha) = i_k 2^k + \cdots + i_0 2^0$. Si le agregamos el símbolo $@$ al final, formamos la nueva palabra $\alpha@$. Como el símbolo $@$ es el primer símbolo de nuestro alfabeto ($a_1$), su índice es $1$. La nueva palabra estructurada es: $a_{i_k} \ldots a_{i_0} a_1$.

Al agregar una letra más a la derecha, la longitud de la palabra crece en 1. Esto provoca que todos los símbolos viejos se desplacen una posición hacia la izquierda, por lo que *el exponente de todas sus potencias aumenta en 1*. El nuevo símbolo ocupará la posición 0. $\#_{\leq}(\alpha@) = i_k 2^{k+1} + \cdots + i_0 2^1 + \mathbf{1 \cdot 2^0}$. Si a toda la primera parte de la suma le sacamos un $2$ como factor común

$$\#_{\leq}(\alpha@) = 2 \cdot (i_k 2^k + \cdots + i_0 2^0) + 1 = \#_{\leq}(\alpha) \cdot \mathbf{2} + \mathbf{1}$$

**Ejercicio 12:** Sea $\Sigma$ un alfabeto finito no vacío y sea $\leq$ un orden total sobre $\Sigma$. Inspírese en el ejercicio anterior para dar una "definición recursiva" de la función $\#^{\leq}$.

Lo que hay que hacer es generalizar lo del ejercicio anterior. Dar la definición recursiva para un alfabeto genérico. Como busco formalizar una recursión, tengo que dejar explícito tanto el caso base como el paso recursivo. El caso base sería la palabra mas chica, como la función da la posición de una palabra en la lista, el caso base va a ser

$$\#_{\leq}(\varepsilon) = 0$$

.

Para el paso inductivo, tengo que generalizar lo del ejercicio anterior, donde le pegabamos un simbolo en específico. La idea sería dar la fórmula para cualquier palabra $\alpha$ a la que le concateno al final un símbolo cualquiera $a_i$. Sabemos entonces lo mismo que antes, que al agregar, todos los símbolos que ya estaban en $\alpha$ se desplazan una posición hacia la izquierda y los exponentes aumentan en $1$, el nuevo símbolo llamemoslo $a_i$ pasa a ocupar la última posición por lo que su aporte va a ser $i \cdot n^0 = i$. Sacamos como factor común el $2$, ya que en ese caso el alfabeto tenía 2 elementos, entonces la formula para el paso recursivo queda como

$$\#_{\leq}(\alpha a_i) = \#_{\leq}(\alpha) \cdot n + i$$

**Ejercicio 13:** Si $\leq$ es un orden total sobre un alfabeto no vacío $\Sigma$, entonces $s^{\leq} = *^{\leq} \circ Suc \circ \#^{\leq}$

Para demostrar la igualdad $s_{\leq} = *_{\leq} \circ Suc \circ \#_{\leq}$, vamos a evaluar el lado derecho de la ecuación aplicándolo a una palabra genérica $\alpha \in \Sigma^*$ y veremos que el resultado se reduce exactamente a la función del lado izquierdo.

1. Tomamos una palabra cualquiera $\alpha \in \Sigma^*$ y le aplicamos la composición de funciones del lado derecho: $(*_{\leq} \circ Suc \circ \#_{\leq})(\alpha)$, se evalua como $*_{\leq}(Suc(\#_{\leq}(\alpha)))$.
2. Llamo $i$ a $i = \#_{\leq}(\alpha)$. Al aplicarle la función $Suc$, que por definición siempre suma 1 a un número natural, la expresión se convierte en: $*_{\leq}(i+1)$.
3. Por definición, para calcular el elemento en la posición $i+1$, debo hacer $*_{\leq}(i+1) = s_{\leq}(*_{\leq}(i))$. Aplicando esta regla a nuestra expresión, nos queda: $s_{\leq}(*_{\leq}(i))$, ahora remplazo $i$ por su valor original ($\#_{\leq}(\alpha)$): $s_{\leq}(*_{\leq}(\#_{\leq}(\alpha)))$.
4. Ahora bien, por lema, la función $\#_{\leq}$ es la inversa exacta de $*_{\leq}$ y por lo tanto $*_{\leq}(\#_{\leq}(\alpha)) = \alpha$.
5. Reemplazando esto en la expresión, se llega a $s_{\leq}(\alpha)$

# 3 Guia 3: Procedimientos efectivos

**Ejercicio 1:** Convensase que el uso de asignaciones puede realizarse usando solo lapiz y papel. Imagine como lo haria en el caso anterior y corrobore que dicho procedimiento es efectivo y ademas computa a $\lambda\alpha[|\alpha|]$.

La idea es convenserse de que esas asignaciones pueden realizarse con papel y lapiz en mano por una persona. Es decir que es un procedimiento efectivo.

- **Inicializar variables** ($A \leftarrow \alpha$ y $B \leftarrow 0$): En la Etapa 1, la persona elige dos lugares en blanco de su papel. A uno le pone la etiqueta $A$ y copia allí todos los símbolos de la palabra de entrada $\alpha$, y al otro lugar le pone $B$ y escribe el número $0$.
- Chequeamos si $A$ no es $\epsilon$, si efectivamente no lo es, sigue a la próxima etapa. Sino se detiene y devuelve $B$.
- Ahora le pedimos a la persona que use el procedimiento de sumarle 1 al número que está en $B$ y llamar a ese resultado $y$, esa asignación es basicamente tachar el $0$ y escribir el nuevo número.
- Ahora **borra el primer elemento de** $\alpha$ tachandola y volvemos al segundo paso del chequeo.

El procedimiento es un ciclo repetitivo entre los pasos 2 y 3 y computa efectivamente la función de longitud.

**Ejercicio 2:** Use los procedimientos $\mathbb{P}_<$ y $\mathbb{P}_{|\ |}$ de los dos ejemplos anteriores para diseñar usando asignaciones un procedimiento que compute a la funcion $\lambda i\alpha[[\alpha]_i]$

**Datos de entrada:** $i \in \omega$ y $\alpha \in \Sigma^*$.

**Etapa 1:** Correr el procedimiento $P_{||}$ con el dato de entrada $\alpha$ y alojar el resultado $y$ de salida en la variable $L$.

$$P_{||}(\alpha) \to y \quad \text{y luego} \quad L \leftarrow y$$

pasar a la etapa $2$.

**Etapa 2:** Ahora el **control del dominio**, si $i = 0$, y a la etapa final.

**Etapa 3:** Correr el procedimiento $P_<$ con el dato de entrada $(L, i)$ esto me diría si el índice es válido y me permite ejecutar el procedimiento. Si el dato de salida es verdadero, quiere decir que $L < i$, ir a la Etapa Final. En caso contrario, seguir a la etapa 4.

**Etapa 4:** Hago las asignaciones iniciales. $A \leftarrow \alpha$ y $C \leftarrow 1$. Luego paso a la etapa 5.

**Etapa 5:** Correr el chequeo de igualdad, si $C = i$, detenerse y dar como salida el primer símbolo de $A$. En caso contrario ir a la siguiente etapa.

**Etapa 6:** Asignamos a $A$ el resultado de tachar el primer símbolo de $A$ y correr el procedimiento $P_{+1}$ con $C$ como dato de entrada y guardar el dato de salida en $C$, es decir $C \leftarrow P_{+1}(C)$. Luego volvemos a la etapa 5.

**Etapa Final:** Ir a la etapa Final.

**Ejercicio 3:** Sea $\mathbb{P}_+$ un procedimiento efectivo que compute a la funcion $\lambda xy\,[x + y]$. Basado en $\mathbb{P}_+$ diseñe un procedimiento efectivo que compute a $\lambda xy\,[x.y]$

Multiplicar $x$ por $y$ es lo mismo que sumar $x$ consigo mismo $y$ veces. Entonces, el procedimiento efectivo para multiplicar dos números naturales $x$ e $y$ es el siguiente:

**Etapa 1:** Inicializar $R \leftarrow 0$ acumulador y $C \leftarrow y$ contador de sumas restantes.

**Etapa 2:** Chequear si $C = 0$. Si es así, detenerse y devolver $R$ como resultado. En caso contrario, seguir a la etapa 3.

**Etapa 3:** Correr el procedimiento $P_+$ con los datos de entrada $R$ y $x$, guardar el resultado en $R$ ($R \leftarrow P_+(R, x)$) y correr el procedimiento $P_{-1}$ con el dato de entrada $C$, guardar el resultado en $C$ ($C \leftarrow P_{-1}(C)$). Luego volver a la etapa 2.

**Ejercicio 4:** Sea $\leq$ es el orden total sobre $\Sigma = \{\blacktriangle, \%\}$ dado por $\blacktriangle < \%$. Usando que $s^{\leq}$ es $\Sigma$-efectivamente computable diseñe un procedimiento efectivo que compute a $*^{\leq} : \omega \to \Sigma^*$

Sea $\leq$ el orden total sobre $\Sigma = \{\blacktriangle, \%\}$ dado por $\blacktriangle < \%$. Se nos pide diseñar un procedimiento efectivo que compute la función de decodificación $*_{\leq} : \omega \to \Sigma^*$, usando el hecho de que su función sucesor $s_{\leq}$ es $\Sigma$-efectivamente computable.

La función $*_{\leq}$ está definida por las siguientes ecuaciones recursivas:

$$*_{\leq}(0) = \varepsilon$$

$$*_{\leq}(x+1) = s_{\leq}(*_{\leq}(x))$$

Dato de entrada: $x \in \omega$.

**Etapa 1:** Asigno inicialmente $W \leftarrow \epsilon$, la palabra actual arrancando desde $0$ y $C \leftarrow x$ el contador de cuántas veces falta aplicar el sucesor. Ir a la etapa 2.

**Etapa 2:** Si $C = 0$, detenerse y devolver $W$ como resultado. En caso contrario, seguir a la etapa 3.

**Etapa 3:** Correr el procedimiento $P_{s_{\leq}}$ con el dato de entrada $W$, guardar el resultado en $W$ ($W \leftarrow P_{s_{\leq}}(W)$) y correr el procedimiento $P_{-1}$ con el dato de entrada $C$, guardar el resultado en $C$ ($C \leftarrow P_{-1}(C)$). Luego volver a la etapa 2.

**Ejercicio 5:** Sea $\Sigma = \{\blacktriangle, \%\}$ y sea $f : D_f \subseteq \omega \times \omega \times \Sigma^* \to \omega$ dada por:

$$\begin{aligned} D_f &= \{(0, 1, \varepsilon), (55, 54, \blacktriangle\%\blacktriangle\%\blacktriangle), (1, 1, \blacktriangle\blacktriangle)\} \\ f(0, 1, \varepsilon) &= 1 \\ f(55, 54, \blacktriangle\%\blacktriangle\%\blacktriangle) &= 2 \\ f(1, 1, \blacktriangle\blacktriangle) &= 3 \end{aligned}$$

Pruebe que $f$ es $\Sigma$-efectivamente computable.

Voy a buscar un procedimiento que funcione como un diccionario. La idea es que el procedimiento compare el dato de entrada con cada uno de los tres elementos válidos del dominio. Si encuentra una coincidencia, corta y da el valor. Pero si el dato de entrada no coincide con ninguno de los 3, significa que no pertenece al dominio por lo que debemos hacer que el procedimiento se quede en un ciclo infinito.

**Datos de entrada:** $x, y \in \omega$ y $\alpha \in \Sigma^*$.

**Etapa 1:** Comparar si $x = 0$, $y = 1$ y $\alpha = \varepsilon$. Si se cumple, detenerse y devolver $1$. En caso contrario ir a la **etapa 2**.

**Etapa 2:** Comparar si $x = 55$, $y = 54$ y $\alpha = \blacktriangle\%\blacktriangle\%\blacktriangle$. Si se cumple, detenerse y devolver $2$. En caso contrario ir a la **etapa 3**.

**Etapa 3:** Comparar si $x = 1$, $y = 1$ y $\alpha = \blacktriangle\blacktriangle$. Si se cumple, detenerse y devolver $3$. En caso contrario ir a la **etapa Final**.

**Final:** Ir a la **Etapa Final**.

*Toda función de dominio finito es trivialmente $\Sigma$-efectivamente computable ya que siempre se va a poder hacer una lista manual de IFs para cada elemento, seguida de un goto al ciclo infinito final.* Va a ser siempre la misma estructura.

**Ejercicio 6:** De un procedimiento efectivo (efectivisable) que compute la funcion $\lambda ix[(x)_i]$

El objetivo es que nos de cuál es el exponente que tiene el $i$-ésimo primo cuando factorizamos el número $x$. En otras palabras, $(x)_i$ es el valor máximo $t$ tal que ese número primo elevado a la $t$ divida a $x$.

El enfoque sería primero buscar cual primo es el que pedimos, luego probamos exponentes, empezando en $0$, y ese número se actualiza a medida que vamos teniendo exitos en el chequeo de si es divisible o no. Es decir, el primo elevado a la $0$ divide? Si divide le sumo uno, sino queda de retorno $0$.

**Datos de entrada** : $i, x \in \omega$.

**Etapa 1:** Si $i = 0$, ir a la etapa **Final**. Si $x = 0$, ir a la etapa **Final**. En caso contrario, ir a la etapa 2.

**Etapa 2:** Hacer las asignaciones iniciales: $T \leftarrow 0$ probador de exponentes y $R \leftarrow 0$ variable que guarda el último exponente que anduvo. Ir a la etapa 3.

**Etapa 3:** Correr el procedimiento $P_{pr}$ con el dato de entrada $i$. Guardar el dato de salida en la variable $P$ ($P \leftarrow P_{pr}(i)$). Ir a la etapa 4.

**Etapa 4:** Correr el procedimiento $P_{pot}$ con el dato de entrada $(P, T)$, guardar el dato de salida en la variable $C$ ($C \leftarrow P_{pot}(P, T)$). Ir a la etapa 5.

**Etapa 5:** Ahora el test de divisibilidad, correr el procedimiento $P_{div}$ con dato de entrada $(C, x)$, esto seria como preguntar si $C$ divide a $x$. Si el resultado es verdadero, entonces pasamos a la etapa 6. En caso contrario, detenerse y dar como resultado el dato de salida a $R$.

**Etapa 6:** Asignar a $R$ el valor de $T$ ($R \leftarrow T$) y asignar a $T$ el resultado de sumarle 1 a $T$ usando el procedimiento $P_{+1}$ ($T \leftarrow P_{+1}(T)$). Luego volver a la etapa 4.

**Etapa Final:** Ir a la etapa Final.

**Ejercicio 7:** Sean

$$f : D_f \subseteq \omega \times \omega \times \Sigma^* \to \omega$$
$$f_1 : D_{f_1} \subseteq \omega \times \Sigma^* \to \omega$$
$$f_2 : D_{f_2} \subseteq \omega \times \Sigma^* \to \omega$$
$$f_3 : D_{f_3} \subseteq \omega \times \Sigma^* \to \Sigma^*$$

(a) Es verdad que $D_{f \circ [f_1, f_2, f_3]} = D_{f_1} \cap D_{f_2} \cap D_{f_3}$?
(b) De una descripcion del dominio de $f \circ [f_1, f_2, f_3]$.
(c) Si $f, f_1, f_2, f_3$ son $\Sigma$-efectivamente computables, entonces $f \circ [f_1, f_2, f_3]$ lo es.

(a) **False**. Esa intersección es únicamente el dominio de la función tupla interna $[f_1, f_2, f_3]$.

(b) La descripción formal es: $D_{f \circ [f_1, f_2, f_3]} = \{(x, \alpha) \in D_{f_1} \cap D_{f_2} \cap D_{f_3} \ : \ (f_1(x, \alpha), f_2(x, \alpha), f_3(x, \alpha)) \in D_f\}$.

(c) **Verdadero**. Supongamos que tenemos el procedimiento así:

1. Primero, tomo la entrada $(x, \alpha)$ e intenta correr las máquinas que computan $f_1, f_2$ y $f_3$, guardando sus resultados en tres variables temporales. (Si el dato no pertenecía a la intersección de sus dominios, alguna de estas máquinas se colgará, haciendo que el programa entero cumpla la regla de colgarse).
2. Si las tres máquinas logran terminar, el programa le pasa esas tres variables como datos de entrada a la máquina que computa $f$.
3. Si el dato arrojado por las máquinas internas no pertenece al dominio de $f$ ($D_f$), la máquina de $f$ entrará en un ciclo infinito (cumpliendo nuevamente el control de dominio). Si pertenece, se detendrá dando el resultado final correcto.

**Ejercicio 8:** V o F o I. Justifique

(a) Si $\mathbb{P}$ es un procedimiento efectivo que computa una funcion no vacia $f : D_f \subseteq \omega \to \omega$ entonces el conjunto de datos de salida de $\mathbb{P}$ es $\omega$

(b) Si $\mathbb{P}$ y $\mathbb{Q}$ son procedimientos efectivos, entonces $\mathbb{PQ}$ lo es.

(c) Denotemos con $e$ a la cantidad de veces que estornudó Alan Turing a lo largo de su vida. Sea $f : \omega \to \omega$, dada por $f(x) = x^e$. Entonces $f$ es $\emptyset$-efectivamente computable.

(d) Si $f$ y $g$ son $\Sigma$-efectivamente computables entonces $f.g$ lo es

(a) **Falso**. El procedimiento efectivo tiene que devolver $f(x)$ en caso de tomar como entrada un $x$ del dominio de $f$ y dicho dominio en este caso es un subconjunto de $\omega$, no obliga que sea necesariamente $\omega$ completo. Entonces dicho procedimiento va a devolver exactamente el mismo conjunto que la imagen de $f$ y no necesariamente va a ser $\omega$ completo.

(b) **Impreciso**. Un procedimiento efectivo pertenece al paradigma filosófico, no es una forma matemática precisa entonces no existe definición de $PQ$.

(c) **Verdadero**. La cantidad $e$ es un número fijo y constante, al ser $e \in \omega$, la función $f(x) = x^e$ es una función bien definida. Y como sabemos que la multiplicación es efectivamente computable, entonces necesariamente existe algún procedimiento que toma $x$ y lo multiplica por sí mismo $e$ veces. Y como su dominio e imagen operan sobre $\omega$, ese procedimiento es $\emptyset$-efectivamente computable.

(d) **Verdadero**. La función producto $f \cdot g$ puede reescribirse usando la función tupla como una composición: $P. \circ [f, g]$, donde $P.$ es la función matemática de la multiplicación ($\lambda xy[x.y]$). Y como las funciones internas son computables junto con la externa de multiplicación, entonces la composición resultante también lo es.

**Ejercicio 9:** Si $f : D_f \subseteq \omega \times \omega \times \{@, \uparrow\}^* \to \omega$ es tal que $D_f$ es finito, entonces $f$ es $\{@, \uparrow\}$-efectivamente computable

Es verdadero, si el dominio es finito podemos hacer como en el ejercicio 5 escribir un procedimiento efectivo que conste de una lista de IFs para cada elemento del dominio, y luego un goto al ciclo infinito final.

**Ejercicio 10:** Sean $f : D_f \subseteq \omega \times \Sigma^* \to \omega$ y $g : D_g \subseteq \omega \times \Sigma^* \to \omega$ funciones $\Sigma$-efectivamente computables tales que $D_f \cap D_g = \emptyset$. Sea $F : D_f \cup D_g \to \omega$ dada por

$$e \to \begin{cases} f(e) & \text{si } e \in D_f \\ g(e) & \text{si } e \in D_g \end{cases}$$

Pruebe que $F$ es $\Sigma$-efectivamente computable.

Como $f$ y $g$ son computables, sabemos que existen dos procedimientos efectivos para ellas: $P_f$ y $P_g$, un enfoque que podríamos darle es:

**Datos de entrada:** $e \in \omega \times \Sigma^*$:

Etapa 1: Ejecutar exactamente un paso o instrucción del procedimiento $P_f$ usando $e$ como entrada.

- Si la máquina $P_f$ indica que se detuvo, detenerse inmediatamente y entregar el dato de salida arrojado por $P_f$.
- Si no se detuvo, pausar la ejecución de $P_f$, guardar el estado en el papel, e ir a la Etapa 2.

Etapa 2: Ejecutar exactamente un paso o instrucción del procedimiento $P_g$ usando $e$ como entrada.

- Si la máquina $P_g$ indica que se detuvo, detenerse inmediatamente y entregar el dato de salida arrojado por $P_g$.
- Si no se detuvo, pausar la ejecución de $P_g$, guardar el estado en el papel, e ir a la Etapa 1.

Para ver que este procedimiento computa bien a $F$, vemos que el dominio es completamente disjunto, o pertenece al de $f$ o al de $g$ pero nunca a ambos. Si el dato pertenece a $D_f$, el procedimiento tiene garantizado detenerse en una cantidad finita de pasos (digamos $k$ pasos). Y no pasó nada con el programa $g$ ya que este solo me lo fue pasando pq nunca va a terminar ya que no pertenece a su dominio. Al fin y al cabo, paralelizando los pasos nos aseguramos de que la máquina no clave el programa entero.

**Ejercicio 11:** $\omega^3 \times \Sigma^{*2}$ es $\Sigma$-efectivamente computable

Para demostrar que un conjunto $S \subseteq \omega^n \times \Sigma^{*m}$ es $\Sigma$-efectivamente computable, hay que demostrar que su **función característica** (denotada como $\chi^S_{\omega^n \times \Sigma^{*m}}$) es una función $\Sigma$-efectivamente computable.

En este ejercicio tengo que ver el conjunto $S = \omega^3 \times \Sigma^{*2}$. Que al verlo bien, $S$ *es exactamente igual a su propio conjunto universal* de referencia. Esto quiere decir que al analizar $\chi^{\omega^3 \times \Sigma^{*2}}_{\omega^3 \times \Sigma^{*2}}$: como cualquier dato de entrada valido que le pasemos a la función permanecerá siempre a dicho conjunto y **nunca va a devolver** $0$. Para absolutamente cualquier entrada posible, siempre será $1$ el valor de retorno.

Se afirma entonces que esa función característica es entonces equivalente a $C^1_{3,2}$, entonces el procedimiento es simple:

**Dato de entrada** : $x_1, x_2, x_3 \in \omega$ y $\alpha_1, \alpha_2 \in \Sigma^*$.

Etapa 1 : Independientemente de cuáles sean los datos de entrada, detenerse inmediatamente y dar como dato de salida el número $1$.

**Ejercicio 12:** Sea $\Sigma = \{▲, \%\}$ y sea

$$S = \{(0, 1, \varepsilon), (55, 54, ▲\%▲\%▲), (1, 1, ▲▲)\}$$

Pruebe que $S$ es $\Sigma$-efectivamente computable.

Ahora tengo que mostrar que su función característica es $\Sigma$-efectivamente computable. Dado que $S$ tiene exactamente $3$ elementos, se puede hacer algo similar al ejercicio 5, con una serie finita de comparaciones:

**Datos de entrada:** $x, y \in \omega$ y $\alpha \in \Sigma^*$

**Etapa 1** : Comparar si $x = 0$, $y = 1$ y $\alpha = \varepsilon$. Si se cumple, detenerse y devolver $1$. En caso contrario ir a la **etapa 2**.

**Etapa 2** : Comparar si $x = 55$, $y = 54$ y $\alpha = ▲\%▲\%▲$. Si se cumple, detenerse y devolver $1$. En caso contrario ir a la **etapa 3**.

**Etapa 3** : Comparar si $x = 1$, $y = 1$ y $\alpha = ▲▲$. Si se cumple, detenerse y devolver $1$. En caso contrario ir a la **etapa 4**.

**Etapa 4** : Detenerse y devolver $0$.

Es similar al ejercicio 5 con la diferencia de que acá estamos computando un conjunto. El dominio de una función característica de un conjunto es todo el universo de datos posibles, por lo que la máquina nunca debería colgarse.

**Ejercicio 13:** Convensase que todo conjunto finito $S \subseteq \omega^n \times \Sigma^{*m}$ es $\Sigma$-efectivamente computable.

La función característica tiene como trabajo evaluar absolutamente cualquier dato del dominio universal y **dar como salida el número 1 si el dato pertenece a $S$, y el número 0 si el dato no pertenece**.

Como el conjunto $S$ es finito, sabemos con total certeza que tiene una cantidad delimitada de elementos, digamos $k$ elementos, a los que podemos llamar $e_1, e_2, \ldots, e_k$. Gracias a esto, siempre es posible construir un procedimiento efectivo que contenga *hardcodeados* a todos y cada uno de estos elementos en una **serie finita de instrucciones lógicas de comparación**.

**Ejercicio 14:** $\{(x, \alpha) \in \omega \times \Sigma^* : |\alpha|^x \text{ es par}\}$ es $\Sigma$-efectivamente computable

Notar que:

- Si el exponente $x = 0$. Por convención en la materia, cualquier número elevado a la 0 da 1 (también $0^0 = 1$). Como el 1 es impar, **para todo $x = 0$ la potencia siempre será impar**.
- Si el exponente $x > 0$:** La potencia $|\alpha|^x$ resultará par **si y solo si la base $|\alpha|$ es un número par**. (*Un número par elevado a cualquier positivo sigue siendo par, y un impar sigue siendo impar*).

Entonces podríamos decir equivalentemente, que la condición de que $|\alpha|^x$ sea par, se reduce a ver si $x > 0$ y que la longitud de $\alpha$ sea par.

**Datos de entrada:** $x \in \omega$ y $\alpha \in \Sigma^*$.

**Etapa 1:** Comparar si $x = 0$. Si se cumple, detenerse y devolver $0$. En caso contrario ir a la etapa 2.

**Etapa 2:** Correr el procedimiento $P_{||}$ con el dato de entrada $\alpha$ y guardar el resultado en la variable $L$. Ir a la etapa $3$.

**Etapa 3:** Como sabemos que la división es computable, la división por hoja que aprendimos en primaria. Podemos correr el procedimiento que nos dé el resto, llamemoslo $P_{\text{resto}}$ con datos de entrada $(L, 2)$. Si el dato de salida es $0$, entonces la longitud de $\alpha$ es par, por lo tanto $|\alpha|^x$ es par, detenerse y devolver $1$. En caso contrario, detenerse y devolver $0$.

**Ejercicio 15:** Sean $S_1, S_2 \subseteq \omega^n \times \Sigma^{*m}$ conjuntos $\Sigma$-efectivamente computables. Entonces $S_1 \cup S_2, S_1 \cap S_2$ y $(\omega^n \times \Sigma^{*m}) - S_1$ son $\Sigma$-efectivamente computables.

- La unión, en la función característica es la disyunción lógica. Entonces podemos hacer un procedimiento que simplemente corra $P_1$ y guarde el resultado en $A$, luego corra $P_2$ y guarde el resultado en $B$, ambos con la misma entrada. Luego en la otra etapa hacemos una comparación: si $A = 1$ o $B = 1$ (es decir, está en alguno de los conjuntos), entonces se detiene y devuelve $1$, sino, si ambos son $0$ (es decir, no está en ninguno de los conjuntos), se detiene y devuelve $0$.
- La intersección, en la función característica es la conjunción lógica. Entonces podemos hacer un procedimiento que simplemente corra $P_1$ y guarde el resultado en $A$, luego corra $P_2$ y guarde el resultado en $B$, ambos con la misma entrada. Luego en la otra etapa hacemos una comparación: si $A = 1$ y $B = 1$ (es decir, está en ambos conjuntos), entonces se detiene y devuelve $1$, sino se detiene y devuelve $0$.
- El último es el complemento, y el complemento es simplemente la negación lógica del procedimiento efectivo de $S_1$, es decir, **sí es $\Sigma$-efectivaemnte comptable**, ya que simplemente armamos un procedimiento donde: Si $P_1$ nos da 1 (estaba en $S_1$), nuestro nuevo programa se detiene y devuelve $0$, y si da $0$ (no estaba en $S_1$), nuestro nuevo programa se detiene y devuelve $1$.

**Ejercicio 16:** Sean $S \subseteq \omega$ y $L \subseteq \{@, \uparrow\}^*$ tales que $(0, \varepsilon) \in S \times L$. Entonces $S$ y $L$ son ambos $\{@, \uparrow\}$-efectivamente computables sii $S \times L$ es $\{@, \uparrow\}$-efectivamente computable

**Primera parte ($\Rightarrow$)** Acá como hipotesis tenemos que $S$ y $L$ son computables, entonces quiere decir que existen procedimientos efectivos $P_S$ y $P_L$ que computan las funciones características de cada uno. Ambas máquinas reciben un dato, nunca se quedan en bucle y sacan solo $1$ o $0$.

Entonces tengo que crear un procedimiento para $\chi^{S \times L}_{\omega \times \{@, \uparrow\}^*}$. Para que un elemento $(x, \alpha)$ pertenezca al producto cartesiano, tiene que cumplirse obligatoriamente que $x \in S$ y $\alpha \in L$. El procedimiento efectivo propuesto es:

**Datos de entrada:** $x \in \omega$ y $\alpha \in @, \uparrow^*$.

**Etapa 1** Correr el procedimiento $P_S$ con el dato de entrada $x$, guardar el resultado en la variable $A$ ($A \leftarrow P_S(x)$) e ir a la etapa 2.

**Etapa 2** Correr el procedimiento $P_L$ con el dato de entrada $\alpha$, guardar el resultado en la variable $B$ ($B \leftarrow P_L(\alpha)$) e ir a la etapa 3.

**Etapa 3** Si $A = 1$ y $B = 1$, entonces $(x, \alpha) \in S \times L$, se detiene y devuelve $1$. En caso contrario, se detiene y devuelve $0$.

**Segunda parte (⇐)** Como hipótesis $S \times L$ es computable. Existe una máquina $P_{S\times L}$ que computa a su función característica. Si le pasamos un par $(x, \alpha)$, nos dice con 1 o 0 si pertenece al producto cartesiano sin colgarse nunca. Y además sabemos que $0 \in S$ y $\epsilon \in L$. Entonces podemos armar los dos procedimientos efectivos fijando el parámetro que ya sabemos que pertenece al conjunto, y así utilizar el procedimiento efectivo del producto cartesiano y tomar en cuenta el valor del conjunto tal que no estemos usando el neutro. Es decir:

- Para $P_S$: Dado un número $x$, correr el procedimiento $P_{S\times L}$ con el dato de entrada $(x, \epsilon)$, y devolver lo que devuelva esa máquina.
- Para $P_L$: Dada una palabra $\alpha$, correr el procedimiento $P_{S\times L}$ con el dato de entrada $(0, \alpha)$, y devolver lo que devuelva esa máquina.

**Ejercicio 17:** Sea $F : \{(x, \alpha) \in \omega \times \Sigma^* : |\alpha|^x \text{ es par}\} \subseteq \omega \times \Sigma^* \to \omega^2 \times \Sigma^*$ dada por $F(x, \alpha) = (x, x^2 + |\alpha|, \varepsilon)$. Diga quienes son $F_{(1)}$,$F_{(2)}$ y $F_{(3)}$. Que funcion es $[F_{(1)}, F_{(2)}, F_{(3)}]$?

- La función $F_{(1)}$ (Primera coordenada): $F_{(1)} : D_F \subseteq \omega \times \Sigma^* \to \omega$ Su regla es $F_{(1)}(x, \alpha) = x$.
- La función $F_{(2)}$ (Segunda coordenada): $F_{(2)} : D_F \subseteq \omega \times \Sigma^* \to \omega$ Su regla es $F_{(2)}(x, \alpha) = x^2 + |\alpha|$.
- La función $F_{(3)}$ (Tercera coordenada): $F_{(3)} : D_F \subseteq \omega \times \Sigma^* \to \Sigma^*$ Su regla es $F_{(3)}(x, \alpha) = \varepsilon$.

Si armamos la función tupla con las funciones componentes que acabamos de definir, lo que estamos haciendo es volver a agrupar los resultados que habíamos separado, evaluándolos sobre la intersección de sus dominios. Por lo tanto, la función resultante reconstruye exactamente la función de partida. Es decir, $[F_{(1)}, F_{(2)}, F_{(3)}] = F$.

**Ejercicio 18:** En la definicion anterior, para el caso $n + m = 1$, quien es $F_{(1)}$?

Según la definición de $F_{(i)}$, si tenemos una función $F$ cuyo codominio es $\omega^n \times \Sigma^{*m}$, la notación $F_{(i)}$ representa a la composición de la proyección $i$-ésima con la propia función $F$, equivalentemente $F_{(i)} = p^i_{n,m} \circ F$. Si la suma de las dimensiones del codominio es 1, significa que nuestra función $F$ no devuelve una tupla de varios elementos, sino que devuelve un único elemento: un número (si $n = 1$ y $m = 0$) o una palabra (si $n = 0$ y $m = 1$). Al componer una proyección trivial con $F$, no estamos modificando ni su dominio ni su resultado. Por lo tanto $F_{(1)} = F$.

**Ejercicio 19:** Pruebe que si $F : D_F \subseteq \omega^k \times \Sigma^{*l} \to \omega^n \times \Sigma^{*m}$, con $k, l, n, m \in \omega$, entonces $F = [F_{(1)}, ..., F_{(n+m)}]$

Para probar que dos funciones son iguales: hay que ver que tengan el mismo dominio y que además para cada elemento de dicho dominio, generen exactamente el mismo resultado.

- Para la primera función, el dominio es simplemente $D_F$. Por otro lado, tenemos una función tupla construida como $[F_{(1)}, \ldots, F_{(n+m)}]$, tenemos como dominio, la intersección de los dominios $D_{F_{(1)}} \cap \cdots \cap D_{F_{(n+m)}}$. Por definición, cada $F_{(i)}$ es la composición de la proyección i-ésima con la función original, es decir $F_{(i)} = p_i^{n,m} \circ F$. Como las proyecciones aceptan cualquier tupla posible que les llegue, el dominio entonces **queda limitado únicamente por el dominio de** $F$, entonces $D_{F_{(i)}} = D_F$.
- Tomo un elemento de entrada válido $e \in D_F$. Al aplicarle la función $F$, esta nos va a dar una tupla de $n + m$ elementos. O sea $F(e) = (y_1, \ldots, y_n, \beta_1, \ldots, \beta_m) = (F_{(1)}(e), \ldots, F_{(n+m)}(e))$. $F_{(1)}(e)$ será el primer elemento de la tupla (es decir, $y_1$), $F_{(2)}(e)$ será el segundo, y así sucesivamente hasta llegar a la última coordenada. Por lo tanto la tupla generada $(F_{(1)}(e), \ldots, F_{(n+m)}(e))$ es igual a la tupla original $(y_1, \ldots, y_n, \beta_1, \ldots, \beta_m)$, que es lo mismo que $F(e)$.

**Lema 1.** *Un conjunto no vacio $S \subseteq \omega^n \times \Sigma^{*m}$ es $\Sigma$-efectivamente enumerable sii hay un procedimiento efectivo $\mathbb{P}$ tal que*

(1) *El conjunto de datos de entrada de $\mathbb{P}$ es $\omega$*
(2) *$\mathbb{P}$ se detiene para cada $x \in \omega$*
(3) *El conjunto de datos de salida de $\mathbb{P}$ es igual a $S$. (Es decir, siempre que $\mathbb{P}$ se detiene, da como salida un elemento de $S$, y para cada elemento $(\vec{x}, \vec{\alpha}) \in S$, hay un $x \in \omega$ tal que $\mathbb{P}$ da como salida a $(\vec{x}, \vec{\alpha})$ cuando lo corremos con $x$ como dato de entrada)*

**Ejercicio 20:** Pruebe el lema anterior. (Hacer el caso $n = 1$, $m = 2$).

Hacer la prueba para $n = 1$ y $m = 2$ es decirnos que nuestro conjunto $S$ está formado por tuplas de un número y dos palabras, es decir $S \subseteq \omega \times \Sigma^{*2}$. Y como es una doble implicación, la demo va a tener dos partes.

**(⇒) Dar el procedimiento** Por hipótesis, sabemos que $S$ es $\Sigma$-efectivamente enumerable y no vacío, es decir, existe una función generadora $F : \omega \to \omega \times \Sigma^{*2}$ tal que todos sus resultados forman exactamente el conjunto $S$ ($Im(F) = S$) y, además, cada una de sus proyecciones de coordenadas ($F_{(1)}$, $F_{(2)}$ y $F_{(3)}$) es de manera independiente $\Sigma$-efectivamente computable.

Como las $3$ funciones son computables, sabemos que existen entonces procedimientos efectivos $P_{F_{(1)}}$, $P_{F_{(2)}}$ y $P_{F_{(3)}}$ que las computan respectivamente y su dominio es $\omega$. Y ahora tenemos que armar el procedimiento para mostrar que *existe y cumple las reglas del lema*.

**Datos de entrada:** $x \in \omega$.

**Etapa 1:** Correr el procedimiento $P_{F_{(1)}}$ con el dato de entrada $x$, guardar el resultado en la variable $X_1$ ($X_1 \leftarrow P_{F_{(1)}}(x)$) e ir a la etapa 2.

**Etapa 2:** Correr el procedimiento $P_{F_{(2)}}$ con el dato de entrada $x$, guardar el resultado en la variable $A_1$ ($A_1 \leftarrow P_{F_{(2)}}(x)$) e ir a la etapa 3.

**Etapa 3:** Correr el procedimiento $P_{F_{(3)}}$ con el dato de entrada $x$, guardar el resultado en la variable $A_2$ ($A_2 \leftarrow P_{F_{(3)}}(x)$) e ir a la etapa 4.

**Etapa 4:** Detenerse y devolver la tupla $(X_1, A_1, A_2)$ como resultado.

Por lo pronto la primera condición se cumple, recibe únicamente $\omega$, siempre para cualquier $x \in \omega$ termina y por último, el dato de salida es exactamente el vector de las tres coordenadas de la función $F(x)$, como la imagen de $F$ recorre todo $S$ por definición, las salidas de dicho procedimiento conforman exactamente a $S$.

**(⇐)** Ahora, tenemos que existe un procedimiento efectivo $P$ que toma un natural $x$, nunca se cuelga, y devuelve tuplas de $\omega \times \Sigma^{*2}$ dando el conjunto $S$. Y lo que tendríamos que hacer es demostrar que $S$ cumple la definición de ser $\Sigma$-efectivamente enumerable armando la función $F$.

Defino $F : \omega \to \omega \times \Sigma^{*2}$ con la regla: $F(x) =$ *dato de salida arrojado por $P$ cuando lo corremos con entrada $x$*. Como $P$ siempre termina y arroja exactamente los elementos de $S$, es obvio que $Im(F) = S$. Pero para que la definición esté completa tengo que ver que las funciones coordenada sean $\Sigma$-efectivamente computables. Para esto, tengo que armar el procedimiento que computa cada una de las coordenadas, pero esto es muy sencillo, ya que simplemente corro el procedimiento $P$ con el dato de entrada $x$, guardo el resultado en una variable temporal, y luego corro la proyección correspondiente a la coordenada que quiero obtener con ese resultado como dato de entrada.

Para $i \in \{1, 2, 3\}$:

**Datos de entrada:** $x \in \omega$.

**Etapa 1:** Correr el procedimiento $P$ con el dato de entrada $x$, guardar la tupla de salida en la variable $V$ ($V \leftarrow P(x)$) e ir a la etapa 2.

**Etapa 2:** Dar como salida final solamente la coordenada $i$-ésima de la tupla $V$.

**Ejercicio 21:** Explique por que es efectivo el procedimiento de (E2)

El procedimiento es efectivo ya que utiliza todas subtareas que ya sabemos que son efectivamente computables.

**Ejercicio 22:** Pruebe que el procedimiento de (E2) tiene conjunto de datos de salida igual a $\omega^2 \times \Sigma^{*3}$.

Tenemos que probar que el conjunto de salida es igual a $\omega^2 \times \Sigma^{*2}$. Es decir, una igualdad de conjuntos, por lo tanto hay que probar ambas inclusiones.

**Conjunto de salida** $\subseteq \omega^2 \times \Sigma^{*2}$ Divido en casos de $x$ y veo cómo se comporta la máquina:

- Si $x = 0$, el programa da como salida la tupla constante $(0, 0, \varepsilon, \varepsilon, \varepsilon)$. Como está formada por dos números naturales y tres palabras vacías, pertenece a $\omega^2 \times \Sigma^{*3}$.
- Si $x \neq 0$, Si $x \neq 0$, el programa calcula dos bajadas $(x)_1, (x)_2$ que extraen los exponentes de los números primos, por definición, estos resultados están en los números naturales. Luego, aplica la biyección $*_{\leq}$ a otras tres bajadas $(x)_3, (x)_4, (x)_5$, lo cual devuelve obligatoriamente tres palabras de $\Sigma^*$. Cuando se junta todo, la 5-upla de salida es $(x_1, x_2, \alpha_1, \alpha_2, \alpha_3)$, la cual pertenece a $\omega^2 \times \Sigma^{*3}$.

$\omega^2 \times \Sigma^{*2} \subseteq$ **Conjunto de salida** Para probar esta inclusión, tenemos que ver que cualquier elemento del producto cartesiano $\omega^2 \times \Sigma^{*2}$, tomemos $e = (A, B, \gamma_1, \gamma_2, \gamma_3)$, puede ser generado por el programa para algún valor de $x$.

La función que lista palabras, $*_{\leq}$, tiene una función inversa $\#_{\leq}$. Por lo tanto, si le aplicamos esta inversa a las tres palabras de nuestra tupla, obtendremos sus correspondientes números de orden: $C = \#_{\leq}(\gamma_1)$, $D = \#_{\leq}(\gamma_2)$, $E = \#_{\leq}(\gamma_3)$.

Entonces ahora con la tupla de 5 números, podemos armar un número $x$ tal que sus bajadas sean exactamente esos números, es decir, $x = 2^A \cdot 3^B \cdot 5^C \cdot 7^D \cdot 11^E$. Al correr el programa con ese $x$, va a generar exactamente la tupla de salida $(A, B, \gamma_1, \gamma_2, \gamma_3)$, que es el elemento del producto cartesiano que habíamos elegido al azar. Por lo tanto cualquier elemento del producto cartesiano puede ser generado por el programa, y se cumple la inclusión.

**Ejercicio 23:** Sea $S = \{(x, \uparrow^x) : x \text{ es par}\}$. Pruebe que $S$ es $\{@, \uparrow\}$-efectivamente enumerable.

Hay que dar un procedimiento efectivo $P$ que reciba como dato de entrada un número natural cualquiera, nunca se cuelgue, y su conjunto global de datos de salida forme el conjunto $S$.

La idea para encarar esto es la siguiente: como el conjunto únicamente acepta números pares en la primera coordenada, nunca podríamos devolver directamente la entrada, entonces usamos el dato de entrada como una semilla por así decirlo, y lo multiplicamos por $2$ para generar un elemento válido ($x = 2n$). Para luego poder armar la segunda coordenada.

**Dato de entrada:** $n \in \omega$.

**Etapa 1** : Multiplicar el dato de entrada por $2$ para generar un número par, es decir, $X \leftarrow 2n$. Ir a la etapa 2.

**Etapa 2** : Asignar a la variable $A$ el valor de $\epsilon$ y a la variable numérica $I$ le asignamos $0$.

**Etapa 3** : Comparar si $I = X$, si es verdadero pasar a la **etapa 5**, en caso contrario pasar a la **etapa 4**.

**Etapa 4** : Concatenar el símbolo $\uparrow$ a la variable $A$ (es decir, $A \leftarrow A \uparrow$), luego incrementar $I$ en $1$ (es decir, $I \leftarrow I + 1$) e ir a la etapa 3.

**Etapa 5** : Detenerse y devolver la tupla $(X, A)$ como resultado.

Ahora para ver que dicho procedimiento cumpla las condiciones del lema, primero que nada la entrada es un único número $n$ contenido en $\omega$. Luego, el único lugar donde podría clavarse le programa es en las etapas $3$ y $4$, ya que en las demas se hacen asignaciones o se detiene. Pero $X$ es siempre un número que calculamos al principio, sabemos que sí o sí, el bucle va a iterar $X$ veces y luego salta a la última etapa.

Ahora bien, cualquiera sea la entrada $n \in \omega$, el programa siempre calcula $X = 2n$ (garantiza que la primera coordenada sea par) y luego concatena el símbolo $\uparrow$ exactamente $X$ veces. Por lo tanto, cualquier salida producida es de la forma $(X, \uparrow^X)$ con $X$ par. Así se ve que toda salida pertenece a $S$.

Sea $e = (k, \uparrow^k)$ un elemento cualquiera de $S$. Por la definición del conjunto, sabemos que $k$ es un número par. Por lo tanto, existe un número natural $n \in \omega$ tal que $k = 2n$. Si tomamos ese número exacto $n$ y lo usamos como dato de entrada para el procedimiento $P$, en la Etapa 1 el programa calculará $X = 2n = k$. Luego armará la palabra con $k$ flechas y se devolvería $(k, \uparrow^k)$, que es exactamente nuestro elemento $e$. Como para todo elemento de $S$ existe una entrada que lo genera, probamos que $S$ está contenida en el conjunto de salida.

Queda probado qe la entrada es un único número, el programa nunca se cuelga, y el conjunto de salida es exactamente $S$. Por lo tanto, se cumple el lema.

**Ejercicio 24:** Sea $S = \{(x, \alpha) \in \omega \times \{@, \uparrow\}^* : \alpha \text{ tiene exactamente } x \text{ ocurrencias de } @\}$. Pruebe que $S$ es $\{@, \uparrow\}$-efectivamente enumerable.

Ahora no podemos simplemente agarrar un $x$ y a partir de el generar un elemento del conjunto de salida, ya que por ejemplo para un $x =$*algo* existen infinitos elementos que pertenecen a $S$.

Hay que buscar entonces una forma de listarlas a todas, la función $*_{\leq}$ toma un número $n$ y devuelve la $n$-ésima palabra del alfabeto. Entonces si varío ese $n$ generando todas las palabras del universo y luego cuento cuantas $@$ tiene cada una, podría armar correctamente el conjunto.

**Dato de entrada:** $n \in \omega$.

**Etapa 1** : Calcular la palabra $\alpha \leftarrow *_{\leq}(n)$ e ir a la etapa 2.

**Etapa 2** : Contar la cantidad de símbolos $@$ que tiene la palabra $\alpha$, guardando el resultado en la variable $A$ e ir a la etapa 3.

**Etapa 3** : Devolver $(A, \alpha)$ como resultado y detenerse.

El procedimiento toma únicamente un elemento $n \in \omega$, además nunca se cuelga. Por lo pronto cualquier salida del procedimiento pertenece a $S$, son todos de la forma $(|\alpha|_{@}, \alpha)$, la cual es exactamente la forma de los elementos de $S$. Ahora bien, si tomo cualquier elemento perteneciente a $S$, con la función inversa a la utilizada, es decir $\#_{\leq}$, podemos saber que número la genera por medio de $n = \#_{\leq}(\text{palabra})$ (obviamente, con la palabra contenida en el alfabeto en cuestión). Esto nos dice que todo $S$ está contenido en el conjunto de salida del procedimiento. Como se cumple la doble inclusión, el conjunto de salida es exactamente $S$.

**Ejercicio 25:** Sean $S_1, S_2 \subseteq \omega^n \times \Sigma^{*m}$ conjuntos $\Sigma$-efectivamente enumerables. Entonces $S_1 \cup S_2$ es $\Sigma$-efectivamente enumerable

Lo que tenemos que lograr es hacer un procedimiento que liste las salidas de $P_1$ y $P_2$, pero como hay infinitos números naturales, no podría simplemente agarrar y decir: primero sacame las de $P_1$ y luego las de $P_2$ ya que podría no terminar nunca. Entonces buscaría repartir las entradas en dos de forma tal que se intercalen, los pares se los paso a $P_1$ y los impartes a $P_2$.

**Dato de entrada** : $x \in \omega$

**Etapa 1** : Fijarse si $x$ es par. Si es par, correr el procedimiento $P_1$ con el dato de entrada $x/2$, detenerse y devolver dicho resultado de salida. En caso contrario, correr el procedimiento $P_2$ con el dato de entrada $(x-1)/2$, detenerse y devolver dicho resultado de salida.

Recibe un número natural $x$, nunca se detiene pq la operacion que hace es finita, y además los procedimientos $P_1$ y $P_2$ siempre paran. Cualquier salida del procedimiento es o una salida de $P_1$ o una salida de $P_2$, por lo tanto pertenece a $S_1 \cup S_2$. Por otro lado, si tomo un elemento cualquiera de $S_1 \cup S_2$, ese elemento es de $P_1$ o de $P_2$. Si es una salida de $P_1$, entonces existe un número natural $n$ tal que al correr $P_1$ con entrada $n$ se obtiene dicho elemento, o sea al correr el procedimiento con la entrada $2n$, se va a ejecutar la parte del programa que corre $P_1$ con entrada $n$, y se va a devolver exactamente el elemento que habíamos elegido. De forma análoga con $P_2$.

**Ejercicio 26:** Sean $S_1, S_2 \subseteq \omega^n \times \Sigma^{*m}$ conjuntos $\Sigma$-efectivamente enumerables. Entonces $S_1 \cap S_2$ es $\Sigma$-efectivamente enumerable. (En el apunte esta probado en forma de lema.)

**Ejercicio 27:** (Explicado en video en granlogico.com) Si $\mathbb{P}$ es un procedimiento efectivo cuyo conjunto de datos de entrada es $\omega \times \Sigma^*$ entonces el conjunto

$$\{(x, \alpha) \in \omega \times \Sigma^* : \mathbb{P} \text{ termina partiendo de } (x, \alpha)\}$$

es $\Sigma$-efectivamente enumerable. Saque como conclusion que el dominio de una funcion $\Sigma$-efectivamente computable es $\Sigma$-efectivamente enumerable.

Primero que nada, si el conjunto **es vacío**, por definición resulta ser $\Sigma$-efectivamente enumerable. Pero si el conjunto es **no vacío**, sabemos que existe al menos un dato de entrada para el cual $P$ sí termina. Tomo uno de ellos $(x_1, \alpha_1)$. Para probar que $S$ es enumerable, hago un procedimiento que reciba un natural, no se cuelgue nunca y que sus salidas sean exactamente $S$.

La idea de tomar un dato que genere $P_S$ y pasarselo a $P$ a ver si termina, no es factible ya que podría quedarse en loop inifinito. Para ello vamos a usar otra táctica:

**Dato de entrada** : $x \in \omega$.

**Etapa 1** : Calcular las primeras $3$ bajadas de $x$: $p = (x)_1$, $q = (x)_2$ y $r = (x)_3$.

**Etapa 2** : Obtengo la palabra correspondiente utilizando $*_{\leq}$, es decir, $\alpha = *_{\leq}(r)$.

**Etapa 3** : Simular el funcionamiento de $P$ pasandole el dato $(x, \alpha)$, pero dejandole ejecutar como máximo $p$ pasos.

**Etapa 4** : Si durante esa cantidad de pasos $P$ logró detenerse, $P_S$ se detiene y da como dato de salida la tupla $(x, \alpha)$.

**Etapa 5** : Si el procedimiento no logró detenerse en $p$ pasos, $P_S$ se detiene y da como dato de salida el comodín $(x_1, \alpha_1)$.

Ahora bien, para ver que el dominio de una función $\Sigma$-efectivamente computable siempre es $\Sigma$-efectivamente enumerable. Vemos que por definición, una función $f$ es $\Sigma$-efectivamente computable, significa que existe un procedimiento $P'$ que la computa, es decir que si le pasas un dato perteneciente a $D_f$, este termina y da $f$(ese dato) y en caso de que no pertenezca al dominio, nunca termina.

Entonces, el dominio de $f$ está conformado por los datos que hacen que la máquina $P'$ se corte y justamente este es el conjunto enumerable, entonces **todo dominio de una función computable es enumerable**.

**Ejercicio 28:** Sea $\Sigma = \{@, !, \%\}$. Si $f : S \subseteq \omega \times \omega \times \Sigma^* \to \omega$ es $\Sigma$-efectivamente computable, entonces $\mathrm{Im}(f)$ es $\Sigma$-efectivamente enumerable

Si el dominio $S$ es vacío, la función nunca devuelve nada, por lo que $Im(f) = \emptyset$, y por definición, es un conjunto $\Sigma$-efectivamente enumerable. Si la función lanza al menos un resultado, quiere decir que existe un elemento en su dominio, tomemoslo como $(z_1, z_2, \gamma) \in S$, para el cual la máquina $P_f$ termina y nos da un resultado válido. Dicho resultado va a ser el comodín: $y_0 = f(z_1, z_2, \gamma)$. Y ahora deberíamos armar el procedimiento que enumere la imagen de $f$.

**Dato de entrada** : $z \in \omega$.

**Etapa 1** : Calcular las $4$ primeras bajadas de $z$: $p = (z)_1$, $q = (z)_2$, $r = (z)_3$ y $s = (z)_4$.

**Etapa 2** : Obtener la palabra correspondiente utilizando $*_{\leq}$, es decir, $\alpha = *_{\leq}(s)$.

**Etapa 3** : Simular el funcionamiento de $P_f$ pasandole el dato $(p, q, \alpha)$, pero dejandole ejecutar como máximo $r$ pasos.

**Etapa 4** : Si durante esa cantidad de pasos $P_f$ logró detenerse, $P_{Im(f)}$ se detiene y da como dato de salida el resultado que dio $P_f$.

**Etapa 5** : Si el procedimiento no logró detenerse en $r$ pasos, $P_{Im(f)}$ se detiene y da como dato de salida el comodín $y_0$.

Tomemos cualquier elemento $Y \in Im(f)$. Por definición de la imagen, sabemos que existe un dato de entrada $(x_1, x_2, \alpha)$ que hace que $P_f$ se detenga y arroje $Y$ en una cantidad finita de pasos $r$. Si le pasamos a nuestra máquina el número $z = 2^p \cdot 3^q \cdot 5^r \cdot 7^{\#_{\leq}(\alpha)}$, la máquina decodificará las variables perfectas, simulará a $P_f$ con el límite de pasos justo y necesario para que termine, y dará el resultado $Y$.

**Ejercicio 29:** Sea $\Sigma = \{@, !, \%\}$. Supongamos $f : S \subseteq \omega \times \Sigma^* \to \Sigma^*$ es $\Sigma$-efectivamente computable. Suponga ademas que $(1, @@) \in S$ y $f(1, @@) = @!!$. Pruebe que el conjunto

$$\{(x, \alpha) \in S : f(x, \alpha) = @!!\}$$

es $\Sigma$-efectivamente enumerable.

Acá ya nos dan el comodín, cuando nos aseguran que $(1, @@) \in S$ y además nos dan el dato de que el conjunto $A$ no es vacío. Entonces el procedimiento sería así:

**Dato de entrada** : $z \in \omega$.

**Etapa 1** : Calcular las primeras $3$ bajadas de $z$: $p = (z)_1$, $q = (z)_2$ y $r = (z)_3$.

**Etapa 2** : Obtener la palabra correspondiente utilizando $*_{\leq}$, es decir, $\alpha = *_{\leq}(r)$.

**Etapa 3** : Simular el funcionamiento de $P_f$ pasandole el dato $(q, \alpha)$ y dejandole ejecutar como máximo $p$ pasos.

**Etapa 4** : Si durante esa cantidad de pasos $P_f$ logró detenerse, $P_A$ se detiene y da como dato de salida el resultado que dio $P_f$.

**Etapa 5** : Si el procedimiento no logró detenerse en $p$ pasos, $P_A$ se detiene y da como dato de salida el comodín $(1, @@)$.

**Ejercicio 30:** Sea $\Sigma = \{@, \&\}$. Supongamos $f : S \subseteq \omega \times \omega \to \omega$ es $\Sigma$-efectivamente computable. Suponga ademas que $(0, 0) \in S$ y $f(0, 0) = 0$. Pruebe que el conjunto

$$\{(x, y) \in S : (f(x, y), y) \in S \text{ y } f(f(x, y), y) = 0\}$$

es $\Sigma$-efectivamente enumerable.

Acá tenemos un comodín, cuando nos dicen que $(0, 0) \in S$ y $f(0, 0) = 0$, sabemos que en $A$ (conjunto que nos piden enumerar), el segundo cálculo haría $f(f(0, 0), 0)$, que pasa a ser $f(0, 0)$, y eso es igual a $0$, entonces el par $(0, 0)$ va a ser el comodín.

**Dato de entrada** $z \in \omega$.

**Etapa 1** : Calculas las primeras $3$ bajadas de $z$: $p = (z)_1$, $q = (z)_2$ y $r = (z)_3$.

**Etapa 2** : Simular el funcionamiento de $P_f$ pasándole el dato $(q, r)$ y dejandole ejecutar como máximo $p$ pasos.

**Etapa 3** : Si durante esa cantidad de pasos $P_f$ se detuvo, guardar ese valor en $V$ ($V \leftarrow P_f(q, r)$) e ir a la etapa 4. En caso contrario, detenerse e ir a la etapa 7.

**Etapa 4** : Simular el funcionamiento de $P_f$ pero ahora pasandole el par $(V, r)$ y dejandole ejecutar como máximo $p$ pasos.

**Etapa 5** : Si durante esa cantidad de pasos $P_f$ se detuvo, guardar ese valor en $W$ ($W \leftarrow P_f(V, r)$) e ir a la etapa 6. En caso contrario, detenerse e ir a la etapa 7.

**Etapa 6** : Comparar si $W = 0$, si es verdadero, detenerse y dar como salida la tupla $(q, r)$, en caso contrario, pasar a la etapa 7.

**Etapa 7** : Detenerse y dar como salida el comodín $(0, 0)$.

**Ejercicio 31:** (Explicado en video en granlogico.com) Si $f : S \subseteq \omega \to \omega$ es $\Sigma$-efectivamente computable, entonces el conjunto

$$\{x \in S : x \text{ es par } x/2 \in S \text{ y } f(x) = f(x/2)\}$$

es $\Sigma$-efectivamente enumerable.

Si $A$ es vacío, es efectivamente enumerable, Si no lo es, sabemos que existe al menos un elemento que cumple todo y pertenece a $A$, este va a ser el comodín $e_0$. Sabemos entonces que $e_0$ es par, que $e_0 \in S$ y $f(e_0) = f(e_0/2)$

**Dato de entrada** : $z \in \omega$.

**Etapa 1** : Calcular las primeras 2 bajadas de $z$: $p = (z)_1$ y $q = (z)_2$.

**Etapa 2** : Comprobar si $q$ es par, si es par, calcular su mitad y guardarla en $K$ ($K \leftarrow q/2$), en caso contrario, detenerse e ir a la etapa 6.

**Etapa 3** : Simular el funcionamiento de $P_f$ pasandole el dato $q$ y dejandole ejecutar como máximo $p$ pasos. Si durante esa cantidad de pasos $P_f$ se detuvo, guardar ese valor en $V$ ($V \leftarrow P_f(q)$) e ir a la etapa 4. En caso contrario, detenerse e ir a la etapa 6.

**Etapa 4** : Simular devuelta el funcionamiento de $P_f$ pasandole el dato $K$ y dejandole ejecutar como máximo $p$ pasos. Si durante esa cantidad de pasos $P_f$ se detuvo, guardar ese valor en $W$ ($W \leftarrow P_f(K)$) e ir a la etapa 5. En caso contrario, detenerse e ir a la etapa 6.

**Etapa 5** : Comparar si $V = W$, si es verdadero, detenerse y dar como salida el número $q$, en caso contrario, pasar a la etapa 6.

**Etapa 6** : Detenerse y dar como salida el comodín $e_0$.

**Ejercicio 32:** Si $f : S \subseteq \omega \to \omega$ es $\Sigma$-efectivamente computable, entonces el conjunto

$$\{x \in S : x^2 \in S \text{ y } f(x) = f(x^2)\}$$

es $\Sigma$-efectivamente enumerable.

Si $A$ es vacío, es trivialmente enumerable. Si no es vacío, existe al menos un elemento que cumpla todo y pertenece a $A$, este va a ser el comodín $e_0$. Sabemos que $e_0 \in S$, $e_0^2 \in S$ y que $f(e_0) = f(e_0^2)$.

**Dato de entrada** : $z \in \omega$.

**Etapa 1** : Calcular las dos primeras bajadas de $z$: $p = (z)_1$ y $q = (z)_2$.

**Etapa 2** : Calculo el cuadrado de $q$ y lo guardo en $K$ ($K \leftarrow q^2$).

**Etapa 3** : Simular el funcionamiento de $P_f$ pasandole el dato $q$ y dejandole ejecutar como máximo $p$ pasos. Si durante esa cantidad de pasos $P_f$ se detuvo, guardar ese valor en $V$ ($V \leftarrow P_f(q)$) e ir a la etapa 4. En caso contrario, detenerse e ir a la etapa 6.

**Etapa 4** : Simular el funcionamiento de $P_f$ pasandole el dato $K$ y dejandole ejecutar como máximo $p$ pasos. Si durante esa cantidad de pasos $P_f$ se detuvo, guardar ese valor en $W$ ($W \leftarrow P_f(K)$) e ir a la etapa 5. En caso contrario, detenerse e ir a la etapa 6.

**Etapa 5** : Comparar si $V = W$, si es verdadero, detenerse y dar como salida el número $q$, en caso contrario, pasar a la etapa 6.

**Etapa 6** : Detenerse y dar como salida el comodín $e_0$.

Tomo un elemento del conjunto, lo llamo $x$, por definición, tanto $f(x)$ como $f(x^2)$ terminan y dan el mismo resultado. Sea $p_1$ la cantidad de pasos que tarda $P_f$ y $p_2$ la cantidad de pasos que tarda $P_f$ con el dato $x^2$. Tomo $p = \text{máx}\{p_1, p_2\}$ como el límite de tiempo que toma para ambos. Como el procedimiento evalúa secuencialmente a todos los números en $\omega$, en algún momento se dará que procesa el número $z = 2^p \cdot 3^x$ y en ese momento se detendrá y dará como resultado $x$, que es un elemento de $A$. Por lo tanto, todo elemento de $A$ pertenece al conjunto de salida del procedimiento, y se cumple la inclusión.

**Ejercicio 33:** Sea $\Sigma = \{@, !, \%\}$. Supongamos $L_1, L_2 \subseteq \Sigma^*$ son $\Sigma$-efectivamente enumerables. Entonces

$$\{\alpha \in L_1 : \text{hay un } \beta \in L_2 \text{ tal que } |\beta| = |\alpha|\}$$

Si $L_1$ o $L_2$ es vacío, entonces ninguna palabra de $L_1$ coincide con longitud de alguna palabra de $L_2$ y por lo tanto $S$ es vacío y por definición es $\Sigma$-efectivamente enumerable. Pero si no es vacío, sabemos que existe al menos una palabra que cumple todo, a la cual la llamaremos testigo $\alpha_0$. Se sabe entonces que $\alpha_0 \in L_1$ y que ademas existe alguna palabra en $L_2$ que coincida con la longitud de esta. Por lo tanto $\alpha_0 \in S$.

**Dato de entrada** $z \in \omega$.

**Etapa 1** : Calcular las primeras 2 bajadas de $z$: $x_1 = (z)_1$ y $x_2 = (z)_2$.

**Etapa 2** : Correr $P_1$ con el dato de entrada $x_1$ y guardamos el resultado en la variable $\alpha$. Como es enumerable sabemos que terminará y devuelve $\alpha \in L_1$, de igual forma con la etapa siguiente.

**Etapa 3** : Correr $P_2$ con el dato de entrada $x_2$ y guardar el dato de salida en $\beta$.

**Etapa 4** : Comparar la longitud de $\alpha$ con la longitud de $\beta$, si son iguales, detenerse y dar como salida $\alpha$, en caso contrario, detenerse y dar como salida el comodín $\alpha_0$.

Tomo cualquier palabra $\alpha \in S$. Por definición de $S$, sabemos que $\alpha \in L_1$ y que existe al menos una palabra $\beta \in L_2$ con su misma longitud. Como $P_1$ y $P_2$ enumeran dichos conjuntos, existen dos números naturales $k_1$ y $k_2$ para los cuales $P_1(k_1) = \alpha$ y $P_2(k_2) = \beta$. Dado que este procedimiento nuestro prueba mediante las bajadas, todas las combinaciones, tarde o temprano, se ingresará el número $z = 2^{k_1} \cdot 3^{k_2}$ para el cual nuestro procedimiento lo procesa, extrae los turnos y devuelve la palabra que buscabamos.

**Ejercicio 34:** Sean $S \subseteq \omega$ y $L \subseteq \{@, \uparrow\}^*$ tales que $(0, \varepsilon) \in S \times L$. Entonces $S \times L$ es $\{@, \uparrow\}$-efectivamente enumerable si y solo si ambos $S$ y $L$ son $\{@, \uparrow\}$-efectivamente enumerables

$\Leftarrow$ **Si $S$ y $L$ son enumerables, entonces $S \times L$ es enumerable.** $S$ y $L$ son $@, \uparrow$-efectivamente enumerables. Como no son vacíos, existen dos procedimientos $P_S$ y $P_L$, que toman un natural, nunca se cuelgan, y devuelven todos los elementos de $S$ y $L$ respectivamente.

**Dato de entrada** $z \in \omega$.

**Etapa 1** : Calcular las primeras 2 bajadas de $z$: $t_1 = (z)_1$ y $t_2 = (z)_2$.

**Etapa 2** : Correr $P_S$ con el dato de entrada $t_1$ y guardar el resultado en la variable $X$. Como es enumerable sabemos que terminará y devuelve $X \in S$, de igual forma con la etapa siguiente.

**Etapa 3** : Correr $P_L$ con el dato de entrada $t_2$ y guardar el dato de salida en $Y$.

**Etapa 4** : Devolver la tupla $(X, Y)$ como resultado y detenerse.

Si tomamos cualquier par $(s, l) \in S \times L$, sabemos que existen turnos $k_1, k_2$ tales que $P_S(k_1) = s$ y $P_L(k_2) = l$. Cuando a nuestro procedimiento le ingresemos el número $z = 2^{k_1} \cdot 3^{k_2}$, desarmará bien el número, dando después la tupla $(s, l)$.

$\Rightarrow$ **Si $S \times L$ es enumerable, entonces $S$ y $L$ son enumerables.** Ahora lo que buscariamos es desarmar la máquina grande que enumera a $S \times L$ en dos independientes, una que enumere únicamente a $S$ y otra a $L$.

**Dato de entrada** $z \in \omega$.

**Etapa 1** : Corrr el procedimiento $P_{S \times L}$ con el dato de entrada $z$ y guardar el resultado en la variable $T$. Como es enumerable sabemos que terminará y devuelve $T \in S \times L$.

**Etapa 2** : Extraer la primera coordenada de $T$ y darla como dato de salida.

Y de manera análoga pero devolviendo la segunda coordenada, obtenemos el procedimiento que enumera a $L$.

**Ejercicio 35:** Sea $f : S \subseteq \Sigma^* \to \Sigma^*$ una funcion $\Sigma$-efectivamente computable. Supongamos que $\varepsilon \in S$ y que $f(\varepsilon) = \varepsilon$. Sea

$$L = \{\alpha \in S : f(\alpha) \in S \text{ y } f(f(\alpha)) = \alpha\}$$

Notar que $\varepsilon \in L$. Pruebe que $L$ es $\Sigma$-efectivamente enumerable

Usando el lema, voy a dar el procedimiento que lo enumere, tome siempre un número en $\omega$, no se detenga y el conjunto de salida sea exactamente el mismo.

**Dato de entrada** $z \in \omega$.

**Etapa 1** : Calcular las primeras $2$ bajadas de $z$: $p = (z)_1$ y $q = (z)_2$.

**Etapa 2** : Obtener la palabra correspondiente utilizando $*_{\leq}$, es decir, $\alpha = *_{\leq}(q)$.

**Etapa 3** : Simular el funcionamiento de $P_f$ pasandole como entrada $\alpha$ y dejandole ejecutar como máximo $p$ pasos. Si durante esa cantidad de pasos logró detenerse, guardar el valor de salida en $X$ y pasar a la siguiente etapa. En caso contrario pasar a la etapa 6.

**Etapa 4** : Simular el funcionamiento de $P_f$ pasandole como entrada $X$ y dejandole ejecutar como máximo $p$ pasos. Si durante esa cantidad de pasos logró detenerse, guardar el valor de salida en $Y$ y pasar a la siguiente etapa. En caso contrario pasar a la etapa 6.

**Etapa 5** : Comparar si $Y = \alpha$, si es verdadero, detenerse y dar como salida $\alpha$, en caso contrario, pasar a la etapa 6.

**Etapa 6** : Detenerse y dar como salida $\epsilon$.

**Ejercicio 36:** V o F o I. Justifique

(a) Denotemos con $e$ a la cantidad de veces que estornudó Alan Turing a lo largo de su vida. Sea $S = \{e\}$. Entonces $S$ es un conjunto $\emptyset$-efectivamente computable

(b) $(2, 4, 6, 8, 10, ...)$ es $\Sigma$-efectivamente enumerable

(c) Si $\mathbb{P}$ es un procedimiento efectivo, entonces $I_{\mathbb{P}}$ es $\Sigma$-efectivamente enumerable

(d) Sea $S \subseteq \omega$ y supongamos $\mathbb{P}$ es un procedimiento efectivo el cual enumera a $S$. Entonces el siguiente procedimiento (con dato de entrada $x \in \omega$) enumera a $\{u \in S : u \text{ es par}\}$:

  (i) Etapa 1: Guardar en la variable $X$ el valor $x$.

  (ii) Etapa 2: Correr $\mathbb{P}$ con dato de entrada $X$ y guardar en la variable $E$ el dato de salida dado por $\mathbb{P}$

  (iii) Etapa 3: Si $E$ es par, dar como salida $E$ y detenerse. Caso contrario aumentar en 1 el valor de $X$ e ir a Etapa 2

(A) **Verdadero**. Sabemos que $S$ es un conjunto finito de un sólo número, y todo conjunto finito es trivialmente computable, ya que existe un procedimiento que simplemente va a comparar la entrada con ese número y devuelve $1$ si es igual o $0$ en caso contrario.

(B) **Impreciso**. El objeto $(2, 4, 6, 8, 10, ...)$ denota un infinitupla, y necesitamos un conjunto para aplicar la definición.

(C) **Impreciso**. La imagen está definida exclusivamente para las funciones, y en este caso estamos hablando de un procedimiento.

(D) **Falso**. No hay seguridad de que la máquina se detenga, puede pasar que $S$ esté formado únicamente por números impares y así nunca cortaría.

## 4 Guía 4: El paradigma de turing

Supongamos ahora que tomamos una palabra $\alpha \in \Gamma^*$ y la distribuimos en la cinta dejando el primer cuadro en blanco y luego poniendo los simbolos de $\alpha$ en los siguientes cuadros. Supongamos ademas que ponemos la maquina en estado $q_0$ y con su cabeza lectora escaneando el primer cuadro de la cinta. Esto lo podemos representar graficamente de la siguiente manera

$$\begin{array}{cccccccc} B & \alpha_1 & ... & \alpha_n & B & B & B & ... \\ \uparrow & & & & & & & \\ q_0 & & & & & & & \end{array}$$

donde $\alpha_1, ..., \alpha_n$ son los sucesivos simbolos de $\alpha$. Supongamos ademas que $a$ ocurre an $\alpha$. Dejamos al lector ir aplicando las clausulas de $M$ para convencerse que luego de un rato de funcionar $M$, la situacion sera

$$\begin{array}{cccccccc} B & \beta_1 & ... & \beta_n & B & B & B & ... \\ \uparrow & & & & & & & \\ q_1 & & & & & & & \end{array}$$

donde $\beta_1 ... \beta_n$ es el resultado de reemplazar en $\alpha$ la primer ocurrencia de $a$ por @.

**Ejercicio 1:** Que sucede cuando $a$ no ocurre en $\alpha$?

**Ejercicio 2:** Para cada uno de los siguientes conjuntos, encuentre una máquina de Turing $M$ (con alfabeto de entrada $\Sigma = \{a, b\}$) tal que $L(M)$ sea dicho conjunto.

(a) $\{\alpha \in \{a, b\}^* : a \text{ ocurre en } \alpha\}$
(b) $\{ab\}$
(c) $\{a^n b^n : n \geq 2\}$ (explicada en video en granlogico.com)
(d) $\{\alpha \in \{a, b\}^* : |\alpha|_a \text{ es par y } |\alpha|_b \text{ es impar}\}$

**Ejercicio 3:** V o F o I, justifique.

(a) Si $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ es una maquina de Turing, entonces $\delta$ es una funcion $(Q \cup \Gamma \cup \{L, K, R\})$-mixta
(b) Si $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ es una maquina de Turing, entonces $D_\delta$ es un conjunto $(Q \cup \Gamma)$-mixto

**Ejercicio 4:** V o F o I, justifique.

(a) Si $d$ es una descripcion instantanea, entonces $Ti(d) = 3-\text{UPLA}$
(b) Si $d$ es una descripcion instantanea, entonces $St(d) = d \cap Q$

**Ejercicio 5:** V o F o I, justifique.

(a) $d \vdash d$, para cada $d \in Des$

(b) Si $\alpha p \beta \nvdash d$ para toda descripción instantánea $d$ entonces $(p, [\beta B]_1) \notin D_\delta$

(c) Si $\delta(p, a) = (p, a, L)$ entonces $pa \nvdash d$ para toda descripción instantánea $d$

(d) Dadas $d, d' \in Des$, se tiene que si $d \vdash d'$, entonces $|d| \leq |d'| + 1$

**Ejercicio 6:** Sea $\Sigma = \{@, \%\}$.

(a) Dar el grafico de una máquina de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ tal que para cada $\alpha \in \Sigma^*$ haya un $q \in Q$ tal que

$$\lfloor q_0 B B \alpha \rfloor \overset{*}{\vdash} \lfloor q B \alpha \rfloor$$

(b) Dar el grafico de una máquina de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ tal que para cada $\alpha \in \Sigma^*$ haya un $q \in Q$ tal que

$$\lfloor q_0 B \alpha \rfloor \overset{*}{\vdash} \lfloor q B B \alpha \rfloor$$

**Ejercicio 7:** Estudie los dos posibles casos de detencion:

(a) estando el cabezal sobre el primer cuadro de la cinta

(b) estando el cabezal en un cuadro que no es el primero

(b) estando el cabezal en un cuadro que no es el primero

**Ejercicio 8:** V o F o I, justifique.

(a) Sea $d \in Des$. Entonces existe una infinitupla $(d_1, d_2, ...)$ tal que $d \vdash d_1 \vdash d_2 \vdash d_3 \vdash d_4 \vdash \cdots$ si y solo si $M$ no se detiene partiendo de $d$

(b) Supongamos que para cada $(p, \sigma) \in Q \times \Gamma$ la tercera coordenada de $\delta(p, \sigma)$ es igual a $L$. Entonces $M$ se detiene partiendo de cada $d \in Des$

**Ejercicio 9:** Para cada uno de los siguientes conjuntos, encuentre una máquina de Turing $M$ tal que $L(M)$ sea dicho conjunto

(a) $\{\alpha \in \{@, \%\}^* : |\alpha|_@ = 2\,|\alpha|_\%\}$

(b) $\{\alpha \in \{@, \%\}^* : \alpha = \alpha^R\}$ (palabras capicuas)

**Ejercicio 10:** Sea $\Sigma = \{@, \%\}$. Dar el grafico de una máquina de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ tal que para cada $\alpha, \beta \in \Sigma^*$, con $|\alpha| = |\beta|$, haya un $q \in Q$ tal que

$$\lfloor q_0 B \alpha \beta \rfloor \overset{*}{\vdash} \lfloor B \alpha q \beta \rfloor \text{ y } \lfloor B \alpha q \beta \rfloor \nvdash d, \text{ para cada } d \in Des$$

**Ejercicio 11:** Sea $\Sigma = \{@, \%\}$. Dar el grafico de una máquina de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ tal que

$$L(M) = \{\beta\beta : \beta \in \{@, \%\}^+\}$$

**Ejercicio 12:** Sea $\Sigma = \{@, \%\}$.

(a) Dar el grafico de una máquina de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ tal que

$$L(M) = \{@^x(@\%)^y : x \neq y \text{ y } x, y \in \mathbf{N}\}$$

(b) Para cada una de los siguientes palabras $\alpha$ dar la sucecion de descripciones instantaneas que parte de $\lfloor q_0 B\alpha \rfloor$.

(i) $\alpha = @@\%$

(ii) $\alpha = \%\%\%$

(iii) $\alpha = \varepsilon$

(iv) $\alpha = @@\%@\%$

(Note que dicha sucesion puede ser finita o infinita)

**Ejercicio 13:** V o F o I, justifique.

(a) Si $q_2$ es un estado final de la máquina $M$, $\delta(q_0, B) = (q_1, B, R)$ y $\delta(q_1, a) = (q_2, b, R)$ entonces $a \in L(M)$.

(b) Si $q_2$ es un estado final de la máquina $M$, $\delta(q_0, B) = (q_1, B, R)$ y $\delta(q_1, a) = (q_2, b, R)$ entonces $b \in L(M)$.

(c) $\alpha \notin L(M)$ si y solo si existe una infinitupla $(d_1, d_2, ...)$ tal que

(i) $St(d_i) \notin F$, para cada $i = 1, 2, ...$

(ii) $\lfloor q_0 B\alpha \rfloor \vdash d_1 \vdash d_2 \vdash d_3 \vdash d_4 \vdash \cdots$

**Ejercicio 14:** Sea $\Sigma = \{a, b\}$. Para cada una de las siguientes funciones $\Sigma$-mixtas dar una máquina de Turing $(Q, \Gamma, \Sigma, \delta, q_0, B, \imath, \emptyset)$ que la compute

(a) $$\begin{array}{rcl} Suc : \omega & \to & \omega \\ n & \to & n+1 \end{array}$$

(b) $$\begin{array}{rcl} Pred : \mathbf{N} & \to & \omega \\ n & \to & n-1 \end{array}$$

(c) $$\begin{array}{rcl} p_2^{1,1} : \omega \times \Sigma^* & \to & \Sigma^* \\ (x, \alpha) & \to & \alpha \end{array}$$

(explicada en video en granlogico.com)

(d) $$\begin{array}{rcl} C_2^{1,1} : \omega \times \Sigma^* & \to & \omega \\ (x, \alpha) & \to & 2 \end{array}$$

(e) $\lambda xy[x + y]$

(f) $\lambda xy[x.y]$

(g) $$\begin{array}{rcl} Q : \omega \times \mathbf{N} & \to & \omega \\ (x, y) & \to & \text{cociente de la division de } x \text{ por } y \end{array}$$

(h) $$\begin{array}{rcl} R : \omega \times \mathbf{N} & \to & \omega \\ (x, y) & \to & \text{resto de la division de } x \text{ por } y \end{array}$$

# 5 Guía 5: Funciones $\Sigma$-recursivas primitivas

**Ejercicio 1:** Justifique con palabras la veracidad del lema anterior (a lo mariposa)

Imaginemos a $f$ como una función maestra que necesita $n$ números y $m$ palabras para ser alimentada, entonces lógicamente tendríamos que darle $n+m$ máquinas mas chicas que procesen los datos iniciales ($f_1, \ldots, f_{n+m}$).

- Las primeras $n$ maquinas $f_1$ a $f_n$ están programadas para devolver **números**, es decir son de tipo $(k, l, \#)$ y esto es lo que necesitabamos pq las primeras $n$ entradas para $f$ son numéricas.
- Las siguientes $m$ máquinas $f_{n+1}$ a $f_{n+m}$ están programadas para devolver **palabras**, es decir son de tipo $(k, l, *)$ y esto es lo que necesitabamos pq las siguientes $m$ entradas para $f$ son palabras.

Ahora bien, como todas las maquinas internas reciben $k$ números y $l$ palabras, y la función grande da un resultado de tipo $s$, es lógico que la maquina completa sea de tipo $(k, l, s)$.

Sobre el **dominio**. Para que toda esa función no se trabe y funcione correctamente al ingresar un dato inicial $(\vec{x}, \vec{\alpha})$, tienen que cumplirse dos condiciones:

- El dato inicial tiene que ser aceptado por **todas** las máquinas internas al mismo tiempo. Si alguna se cuelga, se frena todo. Por eso el dato original tiene que pertenecer a la intersección de todos los dominios de las funciones internas ($\bigcap_{i=1}^{n+m} D_{f_i}$).
- Suponiendo que las maquinas internas lograron terminar su trabajo, van a escupir una tupla de resultados $(f_1(\vec{x}, \vec{\alpha}), \ldots, f_{n+m}(\vec{x}, \vec{\alpha}))$. Para que el proceso sea exitoso, esta tupla de resultados no puede ser cualquier cosa; tiene que ser procesable por $f$. Por ende, esa tupla resultante **debe pertenecer al dominio de** $f$ ($D_f$).

**Lema 2.** *Si $f, f_1, ..., f_r$, con $r \geq 1$, son $\Sigma$-efectivamente computables, entonces $f \circ [f_1, ..., f_r]$ lo es.*

*Proof.* Si $f \circ [f_1, ..., f_r] = \emptyset$, entonces claramente es $\Sigma$-efectivamente computable. Supongamos entonces que $f \circ [f_1, ..., f_r] \neq \emptyset$. Por el lema anterior hay $n, m, k, l \in \omega$ y $s \in \{\#, *\}$ tales que

- $r = n + m$
- $f$ es de tipo $(n, m, s)$
- $f_i$ es de tipo $(k, l, \#)$, para cada $i = 1, ..., n$
- $f_i$ es de tipo $(k, l, *)$, para cada $i = n+1, ..., n+m$

Sean $\mathbb{P}, \mathbb{P}_1, ..., \mathbb{P}_{n+m}$ procedimientos efectivos los cuales computen las funciones $f, f_1, ..., f_{n+m}$, respectivamente. Usando estos procedimientos es facil definir un procedimiento efectivo el cual compute a $f \circ [f_1, ..., f_{n+m}]$ ■

**Ejercicio 2:** Complete la prueba anterior

Sabemos entonces que las maquinas internas toman datos de tipo $(k, l)$, entonces el procedimiento $P_{comp}$ va a ser de la siguiente manera:

**Dato de entrada** : $(x_1, \ldots, x_k, \alpha_1, \ldots, \alpha_l) \in \omega^k \times \Sigma^{*l}$

**Etapa 1** : Para cada $i$ desde $1$ hasta $n$, hacer correr el procedimiento $P_i$ con el dato de entrada $(x_1, \ldots, x_k, \alpha_1, \ldots, \alpha_l)$. En caso de que termine, guardar la salida (que será un número) en la variable $X_i$.

**Etapa 2** : Para cada $i$ desde $n+1$ hasta $n+m$, hacer correr el procedimiento $P_i$ con el mismo dato de entrada $(x_1, \ldots, x_k, \alpha_1, \ldots, \alpha_l)$. En caso de que termine, guardar la salida (que será una palabra) en la variable $A_{i-n}$.

**Etapa 3** : Hacer correr el procedimiento principal $P$ pasándole como dato de entrada la tupla recolectada: $(X_1, \ldots, X_n, A_1, \ldots, A_m)$. En caso de que termine, dar su resultado final como dato de salida y detenerse.

Para ver que el procedimiento funciona, veamos que pasa con las entradas si están o no dentro del dominio:

**Si el dato de entrada sí pertenece al dominio**. Sea un dato inicial $(x_1, \ldots, x_k, \alpha_1, \ldots, \alpha_l) \in D_{f \circ [f_1, \ldots, f_{n+m}]}$. Por definición de dominio de composición, sabemos dos cosas:

- El dato pertenece al dominio de cada una de las $f_i$. Por lo tanto, en las primeras $n+m$ etapas, todas las máquinas $P_i$ lograrán terminar sus cálculos y guardarán exitosamente los resultados en las variables $X_i$ y $A_j$.
- La tupla que resulta de evaluar todas las $f_i$ pertenece al dominio de la función principal $f$. Por lo tanto, en la Etapa Final, cuando a la máquina $P$ le ingrese la tupla recolectada $(X_1, \ldots, X_n, A_1, \ldots, A_m)$, la máquina $P$ logrará terminar y escupirá el resultado correcto.

**El dato no pertenece al dominio**. Sea un dato inicial $(x_1, \ldots, x_k, \alpha_1, \ldots, \alpha_l) \notin D_{f \circ [f_1, \ldots, f_{n+m}]}$. Esto puede ocurrir por dos motivos excluyentes:

- El dato inicial no pertenece al dominio de alguna de las funciones $f_i$. Sea $j$ el índice de la primera función en fallar. Cuando nuestro procedimiento llegue a la etapa $j$, al intentar correr $P_j$, esa máquina se quedará calculando en un bucle infinito. Como nunca devolverá un resultado, nuestro procedimiento queda colgado ahí para siempre.
- El dato inicial sí es procesado por todas las $f_i$, pero la tupla de resultados arrojados $(X_1, \ldots, X_n, A_1, \ldots, A_m)$ no pertenece al dominio de $f$. En este caso, el programa llegará hasta la Etapa Final, pero cuando mande a correr a la máquina $P$, se queda calculando en un bucle infinito.

**Ejercicio 3:** Encuentre ecuaciones que "definan recursivamente" a la funcion $R = \lambda t[2^t]$

Tengo que dar:

- Cuánto vale la función cuando evaluamos el parámetro en $0$ ($R(0)$).
- Cuánto vale la función evaluada en el paso siguiente ($R(t+1)$), pero expresando ese resultado en términos del paso anterior ($R(t)$).

En el caso base, buscamos cuanto vale $R(0)$. $R(0) = 2^0 = 1$ entonces la primera ecuación es $R(0) = 1$.
Ahora queremos saber cuánto vale $R(t+1)$, pero usando el resultado anterior $R(t)$. Planteo: $R(t+1) = 2^{t+1}$. Por las propiedades de la potencia, sabemos que podemos separar eso en una multiplicación: $2^{t+1} = 2 \cdot 2^t = 2 \cdot R(t)$.

**Ejercicio 4:** Encuentre ecuaciones que "definan recursivamente" a la funcion $R = \lambda t[t!]$

En el caso base buscamos $R(0) = 0! = 1$. En el paso recursivo queremos saber cuánto vale la función en el paso siguiente, $R(t+1)$, pero expresándola en términos del paso anterior $R(t)$. Calculamos: $R(t+1) = (t+1)! = t! \cdot (t+1) = R(t) \cdot (t+1)$.

Tambien podemos generalizar pensando que la funcion $R$ depende no solo de un parametro $t$ sino que tiene otras variables. Por ejemplo

(p) $R(0, x_1, x_2, x_3) = x_1 + 2x_3$
(q) $R(t+1, x_1, x_2, x_3) = t + x_1 + x_2 + x_3 + R(t, x_1, x_2, x_3)$

**Ejercicio 5:** Explique con palabras por que las ecuaciones (p) y (q) determinan una unica funcion $R : \omega^4 \to \omega$. Cuanto vale $R(3, 1, 2, 3)$?

Definen una única función $R : \omega^4 \to \omega$ porque definen el valor de la función de manera secuencial y sin ambigüedades.

- La ecuación (p) (el caso base) determina de forma absoluta los valores de $R$ sobre el conjunto $0 \times \omega \times \omega \times \omega$. Es decir, ya sabemos cuánto vale la función para cualquier combinación de variables siempre que $t = 0$.
- Una vez determinados estos valores, la ecuación (q) evaluada con $t = 0$ toma esos resultados previos y determina los valores de $R$ sobre el conjunto $1 \times \omega \times \omega \times \omega$.

- Con los valores de $t = 1$ ya asegurados, aplicamos la ecuación (q) para $t = 1$, lo que determina todos los valores para el conjunto $2 \times \omega \times \omega \times \omega$, y así sucesivamente.

1. **Paso base ($t = 0$)** usamos p

$$R(0, 1, 2, 3) = x_1 + 2x_3 = 1 + 2(3) = 1 + 6 = 7$$

2. **Paso 1 ($t = 0 \Rightarrow t + 1 = 1$)**: usamos q

$$R(1, 1, 2, 3) = t + x_1 + x_2 + x_3 + R(0, 1, 2, 3) == 0 + 6 + 7 = 13$$

3. **Paso 2 ($t = 1 \Rightarrow t + 1 = 2$)**: usamos q

$$R(2, 1, 2, 3) = t + x_1 + x_2 + x_3 + R(1, 1, 2, 3) = 1 + 6 + 13 = 20$$

4. **Paso 3 ($t = 2 \Rightarrow t + 1 = 3$)**: usamos q

$$R(3, 1, 2, 3) = t + x_1 + x_2 + x_3 + R(2, 1, 2, 3) = 2 + 6 + 20 = 28$$

**Ejercicio 6:** Encuentre ecuaciones que "definan recursivamente" a la funcion $R = \lambda t x_1[t + x_1]$, usando la funcion $Suc$.

**Lema 3.** *Sea $S \subseteq \omega \times \Sigma^*$. Entonces $S$ es rectangular si y solo si se cumple la siguiente propiedad:*

(R) *Si $(x, \alpha), (y, \beta) \in S$, entonces $(x, \beta) \in S$*

**Ejercicio 7:** Supongamos $\Sigma = \{\#, \blacktriangle, \%\}$. Use el lema anterior para probar que

$$\{(0, \#\#), (1, \%\%\%)\} \text{ y } \{(x, \alpha) \in \omega \times \Sigma^* : |\alpha| = x\}$$

no son rectangulares

**Ejercicio 8:** Justifique con palabras (a lo mariposa sabia) que la funcion $R(f, g)$ esta bien definida, es decir que dada una $(1+n+m)$-upla $(t, \vec{x}, \vec{\alpha})$ perteneciente a $\omega \times S_1 \times ... \times S_n \times L_1 \times ... \times L_m$, las ecuaciones de (1) y (2) determinan el valor $R(f, g)(t, \vec{x}, \vec{\alpha})$

Primero que nada, entra la tupla $(t, \vec{x}, \vec{\alpha})$,as $n$ variables numéricas y $m$ variables alfabéticas no cambian, lo unico que realmente le da como vida a la recursión es el parámetro natural $t$.

- $R(f, g)(0, \vec{x}, \vec{\alpha}) = f(\vec{x}, \vec{\alpha})$, como $f$ es una función definida para el dominio nuestro, nos da un resultado correcto y único.
- Como $t \in \omega$, sabemos que siempre está a una distancia finita del $0$. La ecuación $R(f, g)(t+1, \vec{x}, \vec{\alpha}) = g(R(f, g)(t, \vec{x}, \vec{\alpha}), t, \vec{x}, \vec{\alpha})$, nos dice que por ejemplo: para calcular el valor del piso $1$; tomamos el resultado seguro del piso $0$, lo combina con lo que teniamos, y produce un resultado nuevo. Es decir, como todo número $t$ se alcanza repitiendo este proceso de sumar $1$ una cantidad finita de veces, el proceso de empujar los resultados hacia adelante aplicando $g$ temrinará irremediablemente construyendo el valor exacto para $t$.Como tant $f$ como $g$ son funciones, es imposible que este proceso genere ambigüedades.

Por lo tanto la dupla de cuaciones funciona como una reacción en cadena que arranca en el dominio y el paso recursivo empuja cada ficha hasta llegar a $t$.

**Ejercicio 9:** Encuentre $f$ y $g$ tales que:

(a) $R(f,g) = C_k^{n,m}$ (con $n, m, k \in \omega$ y $n \geq 1$)
(b) $R(f,g) = \lambda t x_1[t.x_1]$
(c) $R(f,g) = \lambda t x_1 \alpha_1 \alpha_2[t.x_1]$
(d) $R(f,g) = \lambda t x_1[(x_1)^t]$
(e) $R(f,g) = \lambda t[t!]$

**Punto a)** Esta es una función constante que siempre devuelve $k$. Como tiene $n$ variables numéricas de entrada, una de ellas será el parámetro $t$ y las otras $n-1$ serán el resto de variables fijas junto con las $m$ palabras.

- **Caso base ($t = 0$)**: La función debe devolver $k$. Entonces $f$ debe recibir variables fijas (que tiene $n-1$ números y $m$ palabras) y devolver $k$. $f = C_k^{n-1,m}$.
- **Paso recursivo ($t+1$)**: la función $g$ va a recibir la tupla de datos $(A, t, x_2, \ldots, x_n, \alpha_1, \ldots, \alpha_m)$. Queremos que el resultado siga siendo $k$. Como sabemos que el resultado anterior $A$ ya vale $k$, simplemente podemos decirle a $g$ que nos devuelva su primera coordenada. Como $g$ recibe en total $n+1$ variables numéricas y $m$ palabras, usamos una proyección. $g = p_{n+1,m}^1$.

**Punto b)** Esta es la multiplicación. Nuestro parámetro fijo es $x_1$.

- **Caso base ($t = 0$)**: $0 \cdot x_1 = 0$. La función $f$ recibe solo a $x_1$ y debe devolver $0$. $f = C_0^{1,0}$.
- **Paso recursivo ($t+1$)**: $(t+1) \cdot x_1 = t \cdot x_1 + x_1 = A + x_1$. La función $g$ recibe $(A, t, x_1)$ y tiene que sumar el primer parámetro ($A$) con el tercero ($x_1$). Para esto, usamos la función suma $\lambda xy[x+y]$ y la componemos con las proyecciones que extraen el primer y tercer dato. $g = \lambda xy[x+y] \circ [p_{3,0}^1, p_{3,0}^3]$.

**Ejercicio 10:** Explique la forma en la que aplicando los constructores de composicion y recursion primitiva a las funciones del conjunto inicial se puede obtener la funcion $\lambda x_1 x_2 \alpha_1[x_1!]$

# 6 Ejercicios de la Tómbola parcial 1

$C_1^{2,0} \circ [\lambda x[x^2], p_2^{2,0}] = C_1^{2,0}$

lenguajes - Test 1 | lenguajes - Test 2 | 166884

Correct answer: False

- La primera función es $\lambda x[x^2]$, que toma un único número. Su dominio es $\omega$.
- La segunda función es $p_{2,0}^2$, que extrae la segunda coordenada de un par de números. Su dominio es $\omega^2$.
- Intersección: $\omega \cap \omega^2 = \emptyset$.

Al componer $C \circ \emptyset$, el resultado de todo el lado izquierdo es $\emptyset$. Como el lado derecho es $C_{2,0}^1$ (cuyo dominio es $\omega^2$), las funciones son distintas.

$C_1^{1,1} \circ [p_1^{2,2}, p_3^{2,2}] = C_1^{2,2}$

lenguajes - Test 1 | lenguajes - Test 2 | 166197

Correct answer: True

Las dos proyecciones del corchete tienen el mismo dominio ($\omega^2 \times \Sigma^{*2}$).

- El corchete escupe una 2-upla formada por un número y una palabra (resultados de la coordenada 1 y 3).
- Y justo $C_{1,1}^1$ pide exactamente 1 número y 1 palabra.

El resultado final siempre es 1, coinciden dominio y valores ambas funciones.

$\lambda xy\,[Suc(x)] = \lambda yx\,[Suc(x)]$

lenguajes - Test 1 | 166948

Correct answer: False

En las expresiones lambda, el orden importa, por lo tanto son distintas.

$\lambda[0] = C_0^{0,0}$

lenguajes - Test 1 | 167177

Correct answer: True

El dominio de ambas funciones es igual $0$ números y $0$ palabras, y el dominio de las funciones con estas entradas, por definición, es la $0$-upla. Por lo tanto tienen mismo dominio y dan los mismos valores. Si del lado derecho tuviesemos algo distinto como entrada, por ejemplo $C_0^{1,2}$ ya los dominios cambiarían y no se cumple la igualdad.

$D_{\lambda i \alpha[[\alpha]_i]} = \mathbf{N} \times \Sigma^*$

lenguajes - Test 1 166757

Correct answer: False

Your answer: True

El dominio de una función creada con notación lambda está formado por todos los valores que, al ser asignados a las variables, hacen que la expresión interna está definida en forma precisa. Y en la guía tenemos que dados $i \in \omega$ y $\alpha \in \Sigma^*$, se define $[\alpha]_i = i$-esimo elemento de $\alpha$ mientras que dicho $i$ esté entre $1$ y la longitud de $\alpha$, en caso contrario es $\epsilon$.

$(\alpha\beta)^{\curvearrowleft} = \alpha(\beta^{\curvearrowleft})$

lenguajes - Test 1 167029

Correct answer: False

Your answer: True

El caso en que $\beta$ es la palabra vacía ($\varepsilon$) y $\alpha$ tiene un símbolo (supongamos $\alpha = a$).

- Lado izquierdo: $(\alpha\beta) \curvearrowleft$, $a\varepsilon = a$ y como la longitud de $a$ es 1, la definición nos dice que da como resultado la palabra vacía ($\varepsilon$).
- Lado derecho: $\alpha(\beta \curvearrowleft)$, primero aplicamos la operación a $\beta$: $\varepsilon \curvearrowleft$, como la longitud es 0, queda la palabra vacía, y luego al concatenar eso con $a$ da como resultado $a$.

Ambas partes dan resultados distintos, por lo tanto la igualdad es falsa.

Sean $\Sigma = \{!, \%\}$ y $\Gamma = \{\#, \%, |\}$. La funcion

$$\begin{aligned} f : \omega \times \Sigma^* &\rightarrow \Gamma^* \\ (x, \alpha) &\rightarrow \%^x \end{aligned}$$

es $\Sigma$-mixta

lenguajes - Test 1 166053

Correct answer: True

Your answer: False

La definición establece que una función $f$ es $\Sigma$-mixta si cumple estrictamente dos propiedades:

M1 Su dominio $D_f$ está incluido en $\omega^n \times \Sigma^{*m}$ para algunos $n, m \geq 0$.

M2 Su imagen $I_f$ (el conjunto de resultados que realmente devuelve) está incluida totalmente en $\omega$ o totalmente en $\Sigma^*$.

Acá en la función $f(x, \alpha) = \%^x$, el dominio cumple con M1, y en la imagen tenemos el conjunto formado por las palabras producidas al repetir el simbolo $\%$, por lo tanto la función sí es sigma mixta, puesto que la imagen real si está totalmente contenida en $\Sigma^*$.

Dado $x$ un numero natural, $(x)_i$ denota la $i$-esima coordenada de $(x)$

lenguajes - Test 1 166217

Correct answer: True

Your answer: Imprecise

Las bases de la tombola en el puntoi 10, dice que muchas veces un enunciado no aclara que es lo que están denotando ciertos símbolos ya que es bastante obvio puesto que siempre denotan lo mismo, entonces en este caso por esa razón no es impreciso sino que es verdadero.

Sea $\Sigma = \{\%, \&, !\}$ y sea $a$ un simbolo no perteneciente a $\Sigma$. Sea $S = \{(x, \alpha) \in \omega \times \Sigma^* : |\alpha a \alpha| = x\}$. Entonces $S$ no es un conjunto $\Sigma$ -mixto

lenguajes - Test 1 | 167247

Correct answer: False

Your answer: True

Lo mismo que el otro ejercicio, mas allá de que la condición utilice algo de otro lado. Al fin y al cabo está confomado S por un conjunto incluido en $\omega^n \times \Sigma^{*m}$.

Sea $\Sigma$ un alfabeto finito. Sea $S = \{(x, y) \in \omega \times \mathbf{N} : \sqrt[y]{x}$ es irracional$\}$. Entonces $S$ no es un conjunto $\Sigma$-mixto

lenguajes - Test 1 | 166321

Correct answer: False

Your answer: True

El conjunto $N$ está contenido en omega, por lo tanto cumple la definición de conjunto $\Sigma$-mixto de todas formas. No hay nada que escape de la definición, por lo tanto es falso que no sea $\Sigma$-mixto, ya que sí es.

Exercise 1 of 50

(Responder V o F) Siempre que en el apunte o en las guias se define algun concepto, se usa el modo bold para enfatizar cierta parte de dicha definicion

lenguajes - Test 1 | logica - Test 1 | 166321

Dadas funciones cualesquiera $f$ y $g$ la funcion $f \circ g$ esta definida solo cuando $I_g \subseteq D_f$

lenguajes - Test 1 | lenguajes - Test 2 | 166977

Correct answer: False

Dado $x \in \mathbf{N}$, usamos $(x)$ para denotar a la unica infinitupla $(s_1, s_2, \dots) \in \omega^{[\mathbf{N}]}$ tal que

$$x = \prod_{i=1}^{\infty} pr(i)^{s_i}$$

lenguajes - Test 1 | 167079

Correct answer: True

$D_{\lambda i \alpha[[\alpha]_i]} = \mathbf{N} \times \Sigma^*$

lenguajes - Test 1 | 167198

Correct answer: False

$\{(Suc, Pred)\}$ es una funcion

lenguajes - Test 1 | 166360

Correct answer: True

Denotemos con $e$ a la cantidad de veces que estornudo Alan Turing a lo largo de su vida. Sea $S = \{e\}$. Entonces $S$ es un conjunto $\varnothing$ -efectivamente computable

lenguajes - Test 1 | 167304

Correct answer: True